Contents

LLM World-Simulator — Research Learnings

Table of contents

Chapter 1: The Hedge-Fund Harness — the RippleSignal Machine Around the Model

Why the harness, not the model

The ten components, end to end

Component 1 — the entity spine

Component 2 — the point-in-time event panel

Component 3 — graph construction

Component 4 — the propagation engine

Component 5 — mechanism & response functions

Component 6 — scoring: the RippleSignal

Component 7 — the leakage gauntlet

Component 8 — the baseline ladder

Component 9 — backtest & portfolio

Component 10 — runtime & reproducibility

Build order: trust before signal

Honesty ledger

Chapter 2: Code World Models — Learning Environment Dynamics Before RL

What CWM is trying to do

The four training stages

Pretraining

Midtraining: same loss, different data

SFT: putting good behaviours within reach

RL with verifiable rewards

The probability ratio, clipping, and KL

Exploration, entropy collapse, and diversity

Bootstrapping verified trajectories

Benchmarks: CruxEval, LiveCodeBench, SWE-bench

What CWM actually showed

Scaling laws and overtraining for inference

Connection to a financial world model

Chapter 3: Engineering Long Contexts

RoPE: where position enters attention

Why long contexts break RoPE

Extending context: theta and positional scaling

Batching: padding, bucketing, packing, block-diagonal masks

Attention cost: full vs local

GPU memory and the memory-bound problem

FlashAttention

KV cache and GQA

Ring Attention and the Large World Model

Chapter 4: Neural Networks, Universal Approximation, Depth, and Function Approximation

The central idea

$$ \hat f(x)

$$ \phi_j(x)

$$ \hat f(x)

Why do we need nonlinear activation functions?

What a ReLU neuron does

$$ \operatorname{ReLU}(z)

$$ \operatorname{ReLU}(z)

$$ h(x)

$$ h(x)

In higher dimensions, the breakpoint becomes a hyperplane

Why a ReLU network is piecewise linear

$$ \operatorname{ReLU}(w^\top x+b)

$$ \operatorname{ReLU}(w^\top x+b)

How ReLUs create a triangular bump

$$ h(x)

\operatorname{ReLU}(x)

$$ h(x)

x-2(x-1)

$$ h(x)

x-2(x-1)+(x-2)

$$ h(x)

What are x and x_i?

Why multiply bumps by $f(x_i)$?

$$ f(x_i)h_i(x_i)

$$ \hat f(x)

Piecewise-linear interpolation

Why does making the grid finer improve the approximation?

$$ p(x)

$$ f(x)-p(x)

How ReLU represents the whole piecewise-linear function

$$ \boxed{ p(x)

Full example: approximating z^2

$$ m_0

\frac{1-4}{-1-(-2)}

$$ m_1

\frac{0-1}{0-(-1)}

$$ m_2

\frac{1-0}{1-0}

$$ m_3

\frac{4-1}{2-1}

$$ \boxed{ S(z)

Representing the linear term using only ReLUs

$$ \boxed{ z

\operatorname{ReLU}(z)

$$ -3z

The 1D universal approximation proof

$$ p(x)

What the general universal approximation theorem says

$$ \boxed{ \sup_{x\in K} \left| f(x)

Width versus depth

$$ f(x)

Why depth can create exponentially many pieces

$$ T(x)

Why a shallow network may need exponentially more width

$$ f(x)

Multiplication using a ReLU network

$$ \boxed{ xy

$$ \hat m(x,y)

Polynomial approximation

$$ p(x)

Fourier series

RBF networks

$$ \phi_i(x)

$$ \hat f(x)

Dead ReLUs

$$ \frac{d}{dz}\operatorname{ReLU}(z)

Why sums suggest Gaussian behavior

Why products suggest log-normal behavior

$$ \log Y

Gradient norms versus gradient histograms

$$ |g|_2

What gradient histograms can reveal

$$ \boxed{ \text{norm}

\text{overall magnitude} } \qquad \boxed{ \text{histogram}

Topology interlude: open, closed, and bounded sets

Cauchy sequences and completeness

$$ \boxed{ \text{complete}

Compactness

Compact versus complete

The final picture

$$ \phi_i(x)

$$ \phi_i(x)

$$ \boxed{ \text{width}

\text{more nonlinear features in parallel} } \qquad \boxed{ \text{depth}

Chapter 5: The Agent Harness — the Machine Around the Model

The harness is the machine, not the model

Workflow or agent? Autonomy is a cost knob

Component 1 — the agent loop

Component 2 — the tool interface (the ACI)

Component 3 — context management

Component 4 — sub-agents & orchestration

Component 5 — permissions & sandboxing

Component 6 — verification

Component 7 — memory & persistence

Component 8 — observability & evaluation

The meta-harness: the scaffold as a searchable artifact

Build order: a runnable spine before a wide leash

Chapter 6: Injecting an Identity — Textual Inversion, LoRA, and DreamBooth

The problem: a generative model has no notion of "her"

Textual inversion: train the word, freeze the model

LoRA: train a small, reversible slice of the weights

DreamBooth and the catastrophic-forgetting problem

Where identity is actually solved: the image→video pipeline

The open-weight landscape and the economics of "adding a capability"

Chapter 7: Manufacturing the Right Signal — MoE Load Balancing and Lean-Verified Proofs

Part A — The Mixture-of-Experts load-balancing loss

Part B — Verifying mathematics with Lean

Part C — The "Alpha" lineage: AlphaZero → AlphaProof → AlphaEvolve

Chapter 8: Not Forgetting — Catastrophic Forgetting and the SFT → RL → SFT Loop

The forgetting phenomenon: what RL on one skill does to the others

The consolidation dataset: a mixture, never the winners alone

Three knobs for preservation: KL, replay loss, post-RL SFT

Approach 1 — fold preservation into RL

Approach 2 — RL first, then consolidation SFT

Why separating the stages buys exploration

The catch: SFT2 cannot recover what RL never discovered

Is SFT2 mandatory? The iterative distillation view

The tradeoff in one line, and the mental model

Chapter 9: Where to Spend Computation — Depth, Chain-of-Thought, Looping, and the Harness

The four levels where extra computation can live

Level 1 — Transformer depth, and why decode is memory-bound

Level 2 — Chain-of-thought: more tokens, more sequential passes

Level 3 — Latent / recurrent / looped reasoning

Level 4 — Harness, agents, and Recursive Language Models

The synthesis: which computation belongs at which level

The one systems equation to keep

Chapter 10: Scaling Laws — Compute Accounting, Chinchilla, and the Training–Inference Trade-off

The five equations to keep

Where $6ND$ comes from

Why input gradients are computed even though only weights update

Why $6ND$ is only an approximation: projections vs attention mixing

What a scaling law actually predicts

The Chinchilla loss function and the meaning of E

How scaling-law experiments are actually run

Deriving compute-optimal N and D

The "20 tokens per parameter" rule

Kaplan (2020) vs Chinchilla (2022)

Fixed compute: what you lose by training longer

Processed tokens vs unique tokens

Inference-aware scaling — Beyond Chinchilla

The three questions you must not conflate

LLM World-Simulator — Research Learnings

A running notebook for the project: using an LLM as a world-simulator to build an economic-link graph and trade the slow diffusion of shocks across it (the "RippleSignal" strategy). Each chapter is written to be studied from cold.

Table of contents

- **Chapter 1: The Hedge-Fund Harness — the RippleSignal Machine Around the Model**
  - Why the harness, not the model
  - The ten components, end to end
  - Component 1 — the entity spine
  - Component 2 — the point-in-time event panel
  - Component 3 — graph construction
  - Component 4 — the propagation engine
  - Component 5 — mechanism & response functions
  - Component 6 — scoring: the RippleSignal
  - Component 7 — the leakage gauntlet
  - Component 8 — the baseline ladder
  - Component 9 — backtest & portfolio
  - Component 10 — runtime & reproducibility
  - Build order: trust before signal
  - Honesty ledger
- **Chapter 2: Code World Models — Learning Environment Dynamics Before RL**
  - What CWM is trying to do
  - The four training stages
  - Pretraining
  - Midtraining: same loss, different data
  - SFT: putting good behaviours within reach
  - RL with verifiable rewards
  - The probability ratio, clipping, and KL
  - Exploration, entropy collapse, and diversity
  - Bootstrapping verified trajectories
  - Benchmarks: CruxEval, LiveCodeBench, SWE-bench
  - What CWM actually showed
  - Scaling laws and overtraining for inference

- Connection to a financial world model

- **Chapter 3: Engineering Long Contexts**
  - RoPE: where position enters attention
  - Why long contexts break RoPE
  - Extending context: theta and positional scaling
  - Batching: padding, bucketing, packing, block-diagonal masks
  - Attention cost: full vs local
  - GPU memory and the memory-bound problem
  - FlashAttention
  - KV cache and GQA
  - Ring Attention and the Large World Model

- **Chapter 4: Neural Networks, Universal Approximation, Depth, and Function Approximation**
  - The central idea
  - Why we need nonlinear activation functions
  - What a ReLU neuron does; hyperplanes and piecewise linearity
  - How ReLUs create a triangular bump
  - Piecewise-linear interpolation and uniform continuity
  - How ReLU represents a piecewise-linear function
  - Worked example: approximating z^2; the linear-term identity
  - The 1D and general universal approximation theorems
  - Width versus depth; why depth gives exponentially many pieces
  - Multiplication with a ReLU network
  - Other bases: polynomials, Fourier series, RBFs
  - Dead ReLUs; sums→Gaussian, products→log-normal
  - Gradient norms versus histograms
  - Topology: open/closed/bounded, completeness, compactness
  - The final picture

- **Chapter 5: The Agent Harness — the Machine Around the Model**
  - The harness is the machine, not the model
  - Workflow or agent? Autonomy is a cost knob
  - Component 1 — the agent loop
  - Component 2 — the tool interface (the ACI)
  - Component 3 — context management
  - Component 4 — sub-agents & orchestration
  - Component 5 — permissions & sandboxing
  - Component 6 — verification
  - Component 7 — memory & persistence

    - Component 8 — observability & evaluation
    - The meta-harness: the scaffold as a searchable artifact
    - Build order: a runnable spine before a wide leash
- **Chapter 6: Injecting an Identity — Textual Inversion, LoRA, and DreamBooth**
    - The problem: a generative model has no notion of "her"
    - Textual inversion: train the word, freeze the model
    - LoRA: train a small, reversible slice of the weights
    - DreamBooth and the catastrophic-forgetting problem
    - The two-stream training loop — input and output
    - How the diffusion loss is actually computed
    - Why LoRA is the practical winner — and DreamBooth-LoRA the real answer
    - Where identity is actually solved: the image→video pipeline
    - The open-weight landscape and the economics of "adding a capability"
- **Chapter 7: Manufacturing the Right Signal — MoE Load Balancing and Lean-Verified Proofs**
    - Part A — The Mixture-of-Experts load-balancing loss
    - The problem: routing collapse
    - Setup and notation
    - The loss, and where it is computed
    - Why the form $\sum_i f_i P_i$ — the differentiability trick
    - Why the minimum is the balanced state
    - Worked example
    - Router z-loss and auxiliary-loss-free balancing
    - Part B — Verifying mathematics with Lean
    - A verifier that cannot be fooled
    - The tactic REPL, and what LeanDojo does
    - RL with verifiable rewards and expert iteration
    - What the traced data actually trains — and at which stage
    - Part C — The "Alpha" lineage: AlphaZero → AlphaProof → AlphaEvolve
- **Chapter 8: Not Forgetting — Catastrophic Forgetting and the SFT → RL → SFT Loop**
    - The forgetting phenomenon: what RL on one skill does to the others
    - The consolidation dataset: a mixture, never the winners alone
    - Three knobs for preservation: KL, replay loss, post-RL SFT
    - Approach 1 — fold preservation into RL
    - Approach 2 — RL first, then consolidation SFT
    - Why separating the stages buys exploration

    - The catch: SFT2 cannot recover what RL never discovered
    - Is SFT2 mandatory? The iterative distillation view
    - The tradeoff in one line, and the mental model
- **Chapter 9: Where to Spend Computation — Depth, Chain-of-Thought, Looping, and the Harness**
    - The four levels where extra computation can live
    - Level 1 — Transformer depth, and why decode is memory-bound
    - Level 2 — Chain-of-thought: more tokens, more sequential passes
    - Level 3 — Latent / recurrent / looped reasoning
    - Level 4 — Harness, agents, and Recursive Language Models
    - The synthesis: which computation belongs at which level
    - The one systems equation to keep
- **Chapter 10: Scaling Laws — Compute Accounting, Chinchilla, and the Training–Inference Trade-off**
    - The five equations to keep
    - Where $6ND$ comes from
    - Why input gradients are computed even though only weights update
    - Why $6ND$ is only an approximation: projections vs attention mixing
    - What a scaling law actually predicts
    - The Chinchilla loss function and the meaning of E
    - How scaling-law experiments are actually run
    - Deriving compute-optimal N and D
    - The "20 tokens per parameter" rule
    - Kaplan (2020) vs Chinchilla (2022)
    - Fixed compute: what you lose by training longer
    - Processed tokens vs unique tokens
    - Inference-aware scaling — Beyond Chinchilla
    - The three questions you must not conflate

Chapter 1: The Hedge-Fund Harness — the RippleSignal Machine Around the Model

The central lesson of this project so far is counter-intuitive: **the hard part and the durable edge are not the LLM's cleverness, but the software system built around it.** We call that system *the harness*. This chapter defines the harness precisely, then builds it up one component at a time — with the mechanics, the equations, and worked numbers — so that a reader who was not in the sessions could reconstruct both the design and the reasoning behind it.

A *test harness*, in ordinary software, is the scaffolding that runs a piece of code under controlled conditions, feeds it inputs, and checks its outputs — so you can trust the result. Our harness is the same idea scaled to a research strategy: it is everything except the model's cleverness. The pipes that carry data in, the graph the model reasons over, the code that propagates a shock, the layer that scores the answer, and — above all — the battery of tests that decide whether the answer is *real* or an artifact of leakage and luck. The model is the engine; the harness is the car, the road, the crash-test rig, and the referee.

Why the harness, not the model

Any competent fund can now prompt a strong LLM to read a filing and reason about how a shock to one company ripples to its suppliers and customers — that is fast becoming table stakes. What almost nobody does well is the unglamorous 90%: resolving that "AAPL" in a 2019 filing is the same entity as today's Apple, freezing the information set so the model cannot cheat, and proving *after the fact* that the signal is not just a factor everyone already trades, wearing a costume. That 90% *is* the harness, and it is where a small, disciplined team can be better than a large one.

This yields the single most important design principle of the whole project:

> **Trust before signal.** Build the parts that let you *disbelieve* your own results — identity, point-in-time data, the leakage gauntlet — *before* the parts that generate results. A signal you cannot trust is worse than no signal, because it will get you to bet real money on noise.

The failure this prevents is the canonical one: wire up the exciting parts first (LLM + graph + a backtest), see a gorgeous Sharpe ratio, get excited, and only much later discover the backtest quietly used restated data, or the LLM's training cutoff was *after* the test period, or the "signal" was 95% correlated with

a factor you could buy for free. Every one of those is a *harness* bug, not a *model* bug — and building the harness first surfaces them in week 3 rather than after you have raised money on a fiction.

The ten components, end to end

At run time, data flows top to bottom through ten components:

```
0. Entity spine          one identity per company, across all of time
1. PIT event panel       (event, company, timestamp) with an as-of
date on every fact
2. Graph construction    the LLM builds & types the economic-link
graph
3. Propagation engine    a shock spreads across the graph, hop by hop,
with decay
4. Mechanism + response  what KIND of shock, its SIGN, and its
magnitude per link
5. Scoring               turn the effect into one number: the
RippleSignal
6. Leakage gauntlet      the referee — placebos, deflated Sharpe, PBO
(BUILD FIRST)
7. Baseline ladder       the bar the signal must clear + ablations
8. Backtest + portfolio  walk forward in time, size positions, net out
costs
9. Runtime               cheap, cached, seeded, reproducible runs
```

The subtlety that makes this a *harness* and not just a pipeline: the *build* order is not the *run* order. Identity and frozen data come first, but then the gauntlet (component 6) is built *before* the graph/propagation/score it will judge — because the gauntlet is what lets us trust everything above it. The rest of this chapter takes the components in run order; the build order is given at the end.

Component 1 — the entity spine

The **entity spine** (a.k.a. identity master / security master) gives every company one stable internal ID and records every external identifier it ever had — ticker, CIK, LEI, FIGI — together with the date ranges over which each was valid. It is the dictionary that lets the rest of the harness assert that a filing, a price series, a news item, and a graph node all refer to the *same* company even when the surface labels disagree.

This is the least glamorous component and the one that quietly wrecks the most strategies, because company labels are not stable in the ways intuition expects:

- **Tickers get recycled.** When a company dies its ticker is freed and later reassigned to a different company. Join on ticker and you silently glue Company X's 2011 prices onto Company Y's 2020 prices.
- **Companies rename** (Facebook → Meta, `FB` → `META`). Join on symbol and one company splits into two half-histories.
- **Mergers and spin-offs** force a decision about who inherits the history.

- **Every vendor uses a different key** — SEC uses CIK, the legal world uses LEI, Bloomberg uses FIGI, prices arrive keyed by ticker.

The build: assign an opaque internal ID (e.g. `ent_000431`), then attach each external ID as a row carrying `valid_from` / `valid_to`. Every downstream join goes *through* the internal ID and respects those dates. Concretely:

```
{
  "entity_id": "ent_000431",
  "aliases": [
    {"type":"cik",    "value":"0001326801", "from":"2012-05-18",
        "to":null},
    {"type":"ticker", "value":"FB",  "exch":"NASDAQ", "from":"2012-05-
        18", "to":"2022-06-09"},
    {"type":"ticker", "value":"META","exch":"NASDAQ", "from":"2022-06-
        09", "to":null}
  ]
}
// resolve("FB", asof="2019-03-01") -> ent_000431 (Meta, correctly)
// resolve("FB", asof="2024-03-01") -> whoever holds FB THEN, not Meta
```

Public sources are enough for a proof of concept: SEC EDGAR (CIK), GLEIF (LEI, under a CC0 public-domain licence), OpenFIGI (ticker ↔ security). The acceptance test is a deliberately nasty one — a *recycled ticker* — and the spine must return the right entity for every date before anything else is built.

Component 2 — the point-in-time event panel

The **event panel** is the central table: each row is one $(\text{event}, \text{company}, \text{timestamp})$ — e.g. "on 2019-03-04, Company A reported an earnings surprise." Around each event the harness stores the graph as it stood then, the neighbours' state, the evidence the LLM read, its estimated response, its confidence, the consensus estimate, and later the realized outcome. The unit of observation for the entire study is the *event*, not the calendar day.

Point-in-time (PIT) means every fact carries the date it *became knowable*, and you may only read facts knowable on or before your decision date. The clean way to guarantee this is a **bitemporal** store, where each fact has two time axes: *valid time* (the period the fact is about) and *knowledge time* (when you first learned it, and when a correction superseded it). You query "as of knowledge-time D" and receive exactly the world as it looked on D.

Two ways the past lies to you, both fatal if unhandled:

1. **Reporting lag.** Q4 revenue is *about* December but is not *known* until the filing drops in late February. Use December's number in December and you have traded on the future.
2. **Restatement / vintage.** Macro series and even company financials are *revised*. Today's value for 2019 GDP is not the number that existed in 2019. This is the *ALFRED vintage trap*: FRED serves today's revised series; ALFRED

serves the messy number that actually existed on each past date — and only the latter is admissible.

Worked example. Fact: "Acme Q4-2018 EPS = $1.20."

- *Valid time:* Oct–Dec 2018 (the quarter it describes).
- *Knowledge time:* first known 2019-02-21 (press release).
- *Restated:* on 2019-08-09 an amended filing changes it to $1.14.

A decision made on **2019-03-01** must see $1.20 — what existed then — not the "more accurate" $1.14. Accuracy in hindsight is exactly the trap. The bitemporal store returns $1.20 for any query with knowledge-time in $[\text{Feb } 21, \text{Aug } 9)$ and $1.14 only afterward.

Purge and embargo

Because a label at time t depends on a *future* window of returns (our horizon is 4–8 weeks), naive train/test splits leak. Following López de Prado:

- **Purging** drops training examples whose outcome windows overlap the test set.
- **Embargoing** additionally removes a buffer of examples right after the test period, because nearby observations are serially correlated.

Intuitively: a prediction made three weeks before the test window is *still resolving* when the test begins, so its answer is entangled with test-period prices — purge removes it; embargo leaves a gap because "last week's tremor and this week's tremor are the same earthquake." With a 4–8-week horizon, skipping this leaks *by construction*.

The panel is done right when a single call `world_as_of(D)` returns the frozen information set, the store is append-only (corrections are new rows, never overwrites), and splits are by time with purge + embargo. If you cannot answer "what did we know at 9am on this date?" with one call, it is not done.

Component 3 — graph construction

This is the one component where the LLM does the load-bearing work. The **economic-link graph** is directed and *typed*: nodes are companies (entity-spine IDs); edges are relationships with a type and a direction — A `supplies` B, B `is-customer-of` A, A `competes-with` C, A and D `share-input` X, A and E `co-located-in` region R. The typing is the entire point: "the neighbour's stock moved" throws away *why*; a typed edge lets you reason about the mechanism.

Why the LLM, and why *here* specifically: disclosed supply-chain data is thin — firms name only customers above ~10% of revenue, and only the first hop. The interesting links (B's supplier depends on a component from A's region; a tariff on A raises C's input cost two hops away) are buried in the *text* of filings, transcripts, and trade press, and they are non-obvious and cross-industry.

Reading unstructured text and proposing *typed, directional* relationships is what a strong LLM is good at and what tables miss. Asking the LLM "will the stock go up?" is the commoditised, decaying use everyone else runs.

The build proceeds in layers: (1) seed with disclosed facts (≥10% customer disclosures from EDGAR, obvious competitor sets) to get a skeleton you did not hallucinate; (2) have the LLM propose typed edges as structured output `{from, to, type, direction, sign, evidence_span, confidence}`, forcing it to cite the sentence it used so every edge is auditable; (3) constrain rather than trust — reject edges with no evidence span, dedupe against the spine, cap out-degree, keep the confidence. The LLM supplies *structure and sign*; it does not assert magnitudes.

The leakage rule governing this component needs *two* independent guards:

- **Input-side:** feed only documents from the PIT panel with knowledge-time $< t$. Never today's Wikipedia, never a filing that post-dates the event.
- **Weights-side:** use an *open-weight* model with a documented training cutoff earlier than the test period. A closed API model may have read the future during pre-training and you can never prove it did not. This — not pruning — is why the project insists on open weights: for an *auditable cutoff*.

Component 4 — the propagation engine

Once the graph exists, a shock at one company must ripple to its neighbours and *their* neighbours. This is plain, deterministic code, and it must stay that way.

The mechanism is **message passing**, the standard computation on graphs (the engine inside every GNN, and inside AgentTorch). Three steps per hop: each edge computes a **message** from source to destination; each node **aggregates** incoming messages; each node **updates** its state. Run once ⇒ effects travel one hop; run k times ⇒ effects travel k hops.

Worked example, one hop. Shock: A's demand drops 10% (a -0.10 at node A). Edges: A `supplies` B with elasticity 0.6; A `competes-with` C with strength 0.3 (sign flips).

$$m_{A\to B} = -0.10 \times 0.6 = -0.06, \qquad m_{A\to C} = -0.10 \times 0.3 \times (-1) = +$$

B's customer bought less (-0.06 to B's demand); C benefits from a rival's weakness ($+0.03$). Next hop, B passes its (decayed) -0.06 to B's suppliers. Crucially, the *sign* on each edge came from the mechanism classifier (Component 5); the *magnitude* (elasticity) is a parameter calibrated from data — never a number the LLM invented.

The design rule: keep the LLM out of the loop

The LLM supplies **direction and structure**; deterministic code supplies **magnitude and dynamics**. This split is the reason the system is trustworthy: deterministic code does not drift. The same shock through the same graph gives the same numbers every time — debuggable, unit-testable, calibratable. In AgentTorch this is literal: a `float()` cast severs the LLM output from the gradient — the model proposes, the optimiser disposes. (If the propagation is written in a tensor framework, the elasticities and decay become learnable parameters, so calibration is one gradient-descent loop and sensitivity analysis is just reading $\partial\,\text{outcome}/\partial\,\text{param}$ from a single backward pass. That is the genuinely reusable part of the AgentTorch idea — a v1.5 optimisation, not a v1 requirement.)

The failure mode this component owns: propagation explosion

Ripples fade, but the *number of nodes touched* grows geometrically. With average branching factor b and h hops, the reach and the surviving effect are

$$\text{reach}(h) = b^h, \qquad \text{signal}(h) = d^h,$$

where $d \in (0,1)$ is the fraction of effect retained per hop. The quantity that matters is effect *per reached node*:

$$\rho(h) = \frac{\text{signal}(h)}{\text{reach}(h)} = \left(\frac{d}{b}\right)^h.$$

Since $d < 1 < b$ in any realistic graph, ρ decays *geometrically* in h: every extra hop multiplies reach but shrinks effect. **Worked numbers:** with $b = 4$, $h = 3$, $d = 0.6$,

$$\text{reach} = 4^3 = 64, \quad \text{signal} = 0.6^3 = 0.216 = 21.6\%, \quad \rho = \frac{0.216}{64} \approx 0.3$$

Push to $h = 6$ with the same b, d and $\rho \approx (0.15)^6 \approx 1.1 \times 10^{-5}$ — thousands of nodes each nudged by a rounding error. That is not a simulation of an economy; it is confetti. Cap hops at 2–3 and decay hard, so a strong per-node effect reaches a *small, nameable* set of companies you can actually inspect.

Component 5 — mechanism & response functions

The propagation engine needs, per shock: what *kind* it is, which *direction* it pushes a neighbour's fundamentals, and eventually *how much*.

A **mechanism classifier** labels each shock — demand, cost/input, capacity, financing, regulatory, substitution — and, per outgoing edge type, the **sign** it implies for the neighbour's revenue or margin. "Major customer cuts orders" is a *demand* shock: negative along a `supplies→` edge to that customer's supplier, possibly positive along a `competes-with` edge to a rival supplier. Sign matters

more than the raw move: a customer's stock jumping $+20\%$ is ambiguous — more orders (good for suppliers) or cost-cutting / buyback (neutral-to-bad). The customer-momentum signal every fund runs collapses that into one number and hopes; recovering the *mechanism* is the incremental information the whole thesis bets on.

A **response function** is the numeric part: given a shock of a certain type and size arriving at a company, how much does *its* revenue/margin/EPS move — the elasticity the engine multiplies by. Firms react differently (pricing power), so these are per-firm, but most firms have too little history to estimate alone. The fix is **cross-sectional shrinkage** (partial pooling):

$$\hat{\theta}_i = \lambda_i \underbrace{\theta_i^{\text{own}}}_{\text{firm } i\text{'s history}} + (1-\lambda_i) \underbrace{\theta^{\text{prior}}}_{\text{peer/industry}}, \qquad \lambda_i \uparrow \text{ with reliable histor}$$

A data-rich firm has $\lambda_i \to 1$ (trust its own numbers); a sparse firm has $\lambda_i \to 0$ (lean on peers) — exactly where the mispricing often lives. It is the statistically honest way to have an opinion about a firm you have seen only a few times, which is why the plan insists on *cross-sectional* learning rather than one model per name. Keep the shock taxonomy small enough that each type has enough events to validate; an ever-growing taxonomy of 200 bespoke types is overfitting.

Component 6 — scoring: the RippleSignal

The propagation gives an expected change in a company's fundamentals; scoring turns it into a tradable number by comparing to what the market *already* expects. The signal is not "we think EPS will rise" but "we think EPS will rise *more than the analysts do*":

$$\text{RippleSignal}_{i,t} = \Delta\text{EPS}_i^{(\text{harness})} - \Delta\text{EPS}_i^{(\text{consensus})}.$$

Positive ⇒ an expected fundamental surprise the market has not priced ⇒ candidate long; negative ⇒ candidate short. Trading requires an *edge over consensus*, not a correct forecast the market already shares.

The one test the whole strategy exists to pass

The signal is worth building only if it adds information *beyond* the cheap signal everyone has. One cross-sectional regression settles it:

$$r_{i,t+1} = \alpha + \beta_1 \,\text{CustomerMomentum}_{i,t} + \beta_2 \,\text{RippleSignal}_{i,t} + \gamma^\top \text{contr}$$

Everything rides on β_2: is it significant, stable, and the right sign *after* customer momentum and standard factors are already in the regression? If β_2 stays positive and significant with customer momentum sitting right next to it, the harness knows something the simple signal does not — the win. If β_2 collapses

toward 0 once customer momentum is included, all the machinery merely re-derived the free signal the long way round, and the honest move is to use the simple one. This single coefficient is the go/no-go the entire project rides on.

Component 7 — the leakage gauntlet

This is the heart of the harness and the reason the moat is discipline: a battery of tests designed to *kill* the signal. Whatever survives can be trusted. It is built *before* the signal it judges, because building the signal first breeds a love that makes every later test one the signal can pass. The acceptance test for the gauntlet itself is that it *catches a signal you know is fake*.

- **Placebo / negative control.** Run the exact pipeline on input that should produce nothing — a random graph, a shuffled calendar. A pregnancy test that reads positive on tap water is broken. *Pass:* placebo signal ≈ 0.
- **Label-shuffle (permutation test).** Re-pair each prediction with some other event's outcome thousands of times and rebuild the performance distribution. If the real Sharpe sits inside that cloud, the result is luck. *Pass:* real performance in the far tail.
- **Anonymisation.** Strip names/tickers so the model reasons from mechanism, not from recalling how the 2020 crash ended. *Pass:* signal survives.
- **Deflated Sharpe ratio** (Bailey & López de Prado): a Sharpe corrected for *how many strategies you tried* and for non-normal returns. Try 100 variants and the best looks great by chance; the deflated Sharpe asks whether it beats the best of 100 coin-flips. Corollary: pre-commit to *one* strategy so the multiple-testing penalty stays near zero.
- **PBO — probability of backtest overfitting** (Bailey et al.): repeatedly split the trials in half and measure how often the in-sample best underperforms out-of-sample. *Pass:* PBO well below 50%.
- **Fama–MacBeth — count events, not rows.** Estimate the cross-section date by date, then average and test the per-date coefficients. Five thousand stocks in the same crash are close to one fact, not five thousand. *Pass:* significance on the count of *independent periods*.
- **Cutoff & PIT audit.** Automated checks that the model's training cutoff predates every test event and that every fact used has knowledge-time $<$ decision-time. *Pass:* zero facts with a future knowledge-time.

The mindset is *guilty until proven innocent*: treat every positive result as leakage until ruled out. A signal that survives placebo + shuffle + anonymisation

- deflated-Sharpe + PBO + a clean PIT audit is one you genuinely tried to kill and failed — and that, not a pretty equity curve, is what earns real capital.

Component 8 — the baseline ladder

"The agents gave plausible answers" is not a result; the result is beating every simpler, cheaper thing that could explain the same returns. You build those opponents on purpose. Each rung isolates one way the fancy signal could secretly *be* something boring:

Baseline / ablation	What it isolates	If the harness only ties it
Customer momentum	the cheap signal every fund has	fatal — use the cheap one
Centrality-weighted momentum	momentum + basic topology	the graph adds nothing over topology
Standard factor models	value/size/momentum/quality	you built a factor in disguise
Single LLM + RAG	"just ask a smart model with search"	the simulation apparatus is overkill
Graph *without* response functions	value of the mechanism layer	mechanism modelling isn't paying off
Direct-effects-only (1 hop)	value of multi-hop	multi-hop is noise; stay at 1 hop
Shared-analyst factor	co-coverage explaining co-movement	your "links" are just shared attention

The last three are *ablations* of the harness itself — remove one component and re-measure — which is how you attribute performance to a *part* rather than the whole black box, and how you discover which components v1 can drop. Every baseline must run through the *same* gauntlet, PIT panel, and cost model: to believe you beat customer momentum, build the *best honest* customer momentum and beat that.

Component 9 — backtest & portfolio

A signal that predicts returns is not yet a strategy. **Walk-forward** testing respects the arrow of time: fit on an early window, validate on the next, and touch a single most-recent, never-seen window exactly once, at the very end. Never a random split — in time series that shuffles the future into training. The final period is a non-renewable resource: peek and tweak, and its verdict means nothing.

Alpha and risk are kept separate. The optimiser chooses weights w:

$$\max_{w} \ \hat{\alpha}^{\top} w \ - \ \lambda\, w^{\top} \Sigma w \ - \ \gamma\, \mathrm{Turnover}(w) \quad \text{s.t. dollar-, beta-, sector-n}$$

The first term leans into high predicted alpha ($\hat{\alpha}$ = the RippleSignal); the second penalises correlated bets that blow up together (Σ = covariance, λ = risk aversion); the third discourages churn that trading costs would eat (γ =

cost aversion). Neutrality strips out the market and sectors so that profit comes from the ripple insight, not from accidentally being long a rallying sector.

The failure mode this component owns is the **capacity/cost gap**: many paper edges are real but tiny and live in small, illiquid names, so realistic spreads, market impact, and borrow costs erase them. The backtest must charge those costs and report a *net* Sharpe and an honest capacity. A gross-of-cost, microcap-concentrated Sharpe — the trap several cited papers fall into — is not a business.

Component 10 — runtime & reproducibility

The least visible component makes runs cheap and, above all, reproducible. The **archetype trick** (from AgentTorch) keeps LLM calls cheap: instead of one call per company per event (millions), group firms into a few behavioural archetypes, call the LLM once per archetype, then apply the pattern deterministically to every firm in the group — tens of calls, not millions. Compute stops being the constraint; engineering time becomes the real cost, which is exactly where the value is.

A run is **reproducible** only if re-running it later gives the same result, which requires pinning four things in a *run ledger*: (1) model weights + version + cutoff, (2) the exact data vintage (as-of snapshot), (3) all random seeds, (4) the code commit — with LLM outputs cached by `(prompt, model-hash)`. This is a trust feature, not a nicety: if a March result cannot be regenerated bit-for-bit in June, you no longer know whether March was real. It is the PIT discipline of Component 2, turned on your own experiments.

The reassuring corollary on cost: with the archetype trick, realistic POC LLM compute is hundreds to low-thousands of dollars, and public data is ~$300–700/yr. The dominant cost is engineering time on the spine, the panel, and the gauntlet — the components hardest to copy, so time spent there *is* the moat.

Build order: trust before signal

The components run in the order above but are *built* in a different order, which encodes the whole philosophy in a schedule (a 12-week proof of concept):

Weeks	Theme	What gets built
0	data	entity spine + ingest (EDGAR / GDELT / Treasury-ALFRED); pin an open-weight model with a documented cutoff
1–2	rails	the PIT event panel with as-of dates; PIT prices/fundamentals; purge/embargo
3–4	**trust**	**the leakage gauntlet — first**; prove it catches a known-fake signal
5–7	signal	graph + propagation + score on one narrow shock type (major-customer surprises)
8–10	test the moat	the baseline ladder + ablations; is the graph marginal to customer momentum and the shared-analyst factor? does multi-hop beat 1-hop?
11–12	decide	one untouched walk-forward period, net of costs, against the β_2 go/no-go; write the honest verdict, including "no"

The inversion in weeks 3–4 — the referee built before the player — *is* the harness philosophy in one scheduling decision. Everything else is execution. Only after a "go" does live forward-testing begin (the only fully-clean out-of-sample there is), and only then paid data and capacity scaling.

Honesty ledger

Kept deliberately separate, because embellishment has crept into earlier passes:

Claim	Status	Basis / caveat
PIT/bitemporal data, purge & embargo	established	standard quant practice; purge/embargo from López de Prado
Deflated Sharpe, PBO	established	Bailey & López de Prado; Bailey et al. — real, published
Fama–MacBeth "count events not rows"	established	Fama & MacBeth (1973); independence caveat is textbook
Ticker recycling / security-master hazards	established	well-known; GLEIF CC0, EDGAR, OpenFIGI are real
Message passing; LLM-outside-the-gradient; archetype trick; differentiable calibration	verified from source	read from the AgentTorch repo — capability-proof for the *architecture*, not proof of alpha
The specific 10-component decomposition & build order	design choice	our engineering judgement expressing "trust before signal" — sensible, not a theorem
The β_2 incremental-value test as go/no-go	design choice	follows from the stated hypothesis; the threshold is ours to set
"LLM-built links carry alpha beyond customer momentum & the shared-analyst factor"	**hypothesis**	the entire bet — unproven; only the gauntlet + live forward test can settle it

> **Key takeaway.** The harness is not scaffolding *for* the edge; in this project the harness *is* the edge. The LLM's graph-building is necessary but copyable; the durable advantage is the discipline that lets you trust — or, more often, correctly distrust — what the model produces. Build the referee before the player, freeze the past before you query it, and make the signal beat the cheap version of itself before you believe a single number.

Chapter 2: Code World Models — Learning Environment Dynamics Before RL

Chapter 1 argued that our strategy needs a *world model* of the economy — a component that maps `state + action → next state`. It is worth understanding how the frontier labs are building world models in a domain where the ground truth is cheap and verifiable: **code**. Meta's *Code World Model* (CWM) is the clearest worked example, and studying it pays off twice — once as an education in how modern coding agents are actually trained (pretraining → midtraining → SFT → RL), and once as a template we can transplant to finance (the last section). This chapter teaches the whole pipeline from cold, with the RL mathematics that makes it work.

What CWM is trying to do

CWM asks a deceptively simple question:

> Can a coding model become a better *agent* if it first learns how a computer environment changes when actions are taken?

A normal code model mostly sees *static* code. It learns that after `x.append(...)` some particular code tends to come next — it learns what code *looks like*. CWM additionally trains on actual execution trajectories:

```
state:   x = [1, 2]
action:  x.append(3)
state':  x = [1, 2, 3]
```

so it learns what code *does*, not just what it looks like. The "world" in Code World Model is the computational environment: Python variables and program state, files, repositories, terminal commands, tests, compiler/runtime errors, and tool outputs. The fundamental relation being modelled is

$$\text{current state} + \text{action} \longrightarrow \text{next state}.$$

The four training stages

The pipeline is easy to remember as **Pretraining → Midtraining → SFT → RL**. The first three are still fundamentally next-token prediction trained with cross-entropy; **RL is where the objective changes** from imitation to optimising a reward.

Pretraining

Ordinary LLM training. The model sees text, documentation, math, and large code corpora, receives token sequences $x_1, x_2, \ldots, x_T$, and predicts the next token $P(x_t \mid x_{<t})$ under the cross-entropy loss

$$L = -\sum_t \log P(x_t \mid x_{<t}).$$

The goal is broad competence: language, programming syntax, algorithms, code patterns, and reasoning primitives. At this stage the model mostly learns what code and language *look like*.

Midtraining: same loss, different data

Midtraining simply means continuing pretraining *after* the base model exists but *before* SFT/RL — it is "mid" because it sits between general pretraining and post-training. The loss can be *exactly* the same next-token cross-entropy; what changes is the **data**. This is where the world-model idea enters, through two especially important data types.

Python execution traces. Real code is executed and the interpreter's state is serialised into tokens. For

```
x = 2
x += 4
x *= 3
```

the trace is `x = 2 → (x += 4) → x = 6 → (x *= 3) → x = 18`. To predict the next state correctly the model benefits from internalising that `x *= 3` means $x_{\text{new}} = 3\,x_{\text{old}}$. This teaches *computational semantics* — the difference between "what does the code look like?" (syntax) and "what happens when it runs?" (semantics).

ForagerAgent trajectories. ForagerAgent is a separate coding agent that interacts with real software environments — run `pytest`, see a failure, read the file, edit line 42, rerun, tests pass. It is *not* used because it is a perfect teacher; it is used because it generates useful trajectories of the form

$$\text{observation} \rightarrow \text{action} \rightarrow \text{real environment response}.$$

Even a *bad* action is valuable: a broken edit that raises a new exception still teaches something true about the environment's dynamics. The reason to use an intelligent agent rather than random commands is that a smart policy visits *useful* states (bug → inspect traceback → inspect file → patch → rerun) instead of wasting data on `ls; pwd; cat random_file`. So ForagerAgent is best understood as a **smart data-collection policy**.

The key conceptual point: midtraining needs no special "world-model loss." You serialise `STATE / ACTION / STATE` sequences, tokenise, and train with plain $-\log P(\text{next token})$. That teaches dynamics *because the correct next tokens depend on the actual dynamics* — to reliably emit `x = 6` you need an internal representation of what `x += 4` does. **Same loss, different training signal.**

SFT: putting good behaviours within reach

After midtraining the model understands a lot about computation but still needs to learn how a useful assistant should *behave*. Supervised fine-tuning shows curated demonstrations — "the auth test fails on `None` email; inspect the test, find the null-handling branch, fix it" — again with cross-entropy, but now the targets are *desirable* assistant responses. The distinction from midtraining is purpose: **midtraining teaches what the world does; SFT teaches what a good agent does.**

Why SFT must precede RL is a probability-mass argument. Suppose a genuinely good reasoning strategy currently has probability $\approx 10^{-6}$. RL can only reinforce what it *samples*, so it may never see that strategy. If SFT demonstrates it repeatedly and lifts its probability to, say, 0.05, RL now has a realistic chance to explore and refine it. The mental model:

> SFT puts useful behaviours into the model's reachable search space; RL then discovers which of those behaviours actually work best.

RL with verifiable rewards

RL differs because the *current* model now actively interacts with a real environment, and the trajectory depends on what the current model chooses:

```
Task: fix issue #814
model: run pytest        → env: 17 pass, 2 fail
model: open parser.py    → env: [file contents]
model: edit parser.py    → env: file updated
model: run pytest        → env: 19 pass
verifier: correct? → reward
```

This is unlike midtraining, where the trajectory was already recorded in the dataset. Crucially, the model does **not** grade itself. Reward comes from an *externally verifiable* source — benchmark authors, repository developers, contest judges, hidden test suites: `model code → execute → run tests → reward`. Tests can be incomplete (so reward hacking is possible), but this is far stronger than subjective self-grading.

The LLM already is a policy. Before RL the model produces a distribution over the next token, $\pi_\theta(x_{t+1} \mid x_{\le t})$. During RL we reinterpret it as a policy $\pi_\theta(a_t \mid s_t)$, where the action is the next generated token and a whole answer has

probability $P(y_1, \ldots, y_T) = \prod_t P(y_t \mid y_{<t})$. No separate policy network is needed — the LLM is already a stochastic policy. RL then nudges the weights so that tokens/actions on *better* trajectories become more likely in similar situations. CWM uses a GRPO/PPO-style method.

The probability ratio, clipping, and KL

Where the ratio comes from. The rollout was generated by an older policy π_{old}, but after gradient steps we have π_θ. Because the samples came from the old distribution, PPO reweights them with an importance-sampling ratio

$$r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

$r_t = 1.2$ means the action is now 20% more likely than when it was sampled; $r_t = 0.5$ means half as likely. Vanilla REINFORCE with completely fresh samples does not need this correction — it appears precisely because we reuse experience from an older policy.

Why clip. If a successful action jumped from probability 0.01 to 0.9 in one step, that would be an enormous, potentially destabilising update. PPO uses

$$L^{\text{clip}} = \min\Big(r_t A_t,\ \text{clip}(r_t,\, 1-\epsilon,\, 1+\epsilon)\, A_t\Big),$$

where A_t is the advantage. The object being clipped is the *probability ratio* — not the raw gradient and not the advantage. Once a sampled action has already moved far enough in the desired direction, PPO stops rewarding it for moving further on this batch, making updates conservative.

Clipping vs KL. A KL penalty $R - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ says "don't let the *whole policy* drift far from a reference model." Clipping says "don't let *sampled-action probabilities* move too much this update." Similar stabilising motives, not identical; systems use clipping only, KL only, or both. CWM relied on its GRPO/PPO-style stabilisation rather than an explicit standard KL penalty.

Why importance sampling has high variance. Estimating $E_p[f(x)]$ from samples of q uses $E_q\big[\frac{p(x)}{q(x)} f(x)\big]$ with weight $w(x) = p(x)/q(x)$. The variance carries a term like $E_q[w(x)^2 f(x)^2]$, so if $q(x)$ is tiny where $p(x)$ is large, $w(x)$ blows up and a single rare sample dominates the estimator. Clipping the ratio is exactly what prevents extreme weights from producing extreme updates.

The state-distribution caveat. The ratio only corrects the *action* distribution. The old policy also visited states with density $d^{\pi_{\text{old}}}(s)$, while a very different new policy would visit $d^{\pi_\theta}(s)$ — which the ratio does not fix. Hence PPO is kept *near-on-policy*: generate fresh rollouts, make a few small updates, regenerate, repeat — you do not keep training forever on stale trajectories.

Exploration, entropy collapse, and diversity

Pure reward maximisation pushes probability toward whatever scores highest. If strategy A succeeds 80% and B 20%, RL shifts mass toward A and away from B, C, …, and policy entropy declines — **entropy (or policy) collapse**. This is dangerous because a strategy that is *initially* rare but *could* become excellent with more learning can be pushed toward zero too early, triggering a feedback loop:

```
low probability → sampled less → less training signal → even lower
probability
```

Countermeasures include PPO clipping, KL penalties, multiple rollouts per prompt, exploration temperature, diverse training tasks, explicit entropy bonuses, mixing supervised data back in, and recycling verified successes as future SFT data. An explicit entropy term gives $J = E[R] + \alpha\, H(\pi)$, but in LLMs naive *token* entropy is not the right notion of useful exploration — you want diversity of *valid strategies* (a different valid proof, a different correct algorithm), not random irrelevant tokens. So modern LLM RL does not universally use a simple entropy bonus.

This is also why **RL cannot discover everything**: it can only reward what gets sampled. If the best solution has probability 10^{-10} you may never see it. External verification answers "was this sampled solution good?" but not "what great solution did we never sample?" — which is why SFT, exploration, rejection sampling, search, and bootstrapping all matter.

Bootstrapping verified trajectories

A powerful self-improvement loop:

```
current model → generate many solutions → external verifier keeps the
good ones
→ use those as SFT data → train a better model → generate better
solutions → (RL refines)
```

This is the family that includes rejection sampling, expert iteration, self-training, and distillation. The indispensable safeguard is the **external verifier**; without it you would be training the model on its own mistakes.

Benchmarks: CruxEval, LiveCodeBench, SWE-bench

Three benchmarks probe increasingly different abilities, and the percentage in each roughly means "fraction of problems solved":

- **CruxEval** — small-program *execution reasoning*: given `f` and an input, predict the output. Tests whether the model understands what code actually does.
- **LiveCodeBench** — competitive-programming coding: write a program, hidden tests run it; solved when the submission passes. Tests algorithmic

coding.

- **SWE-bench Verified** — real repository issues: understand the issue → inspect the repo → edit code → run tests → produce a correct patch; resolved when the patch passes evaluation. Tests real repository-level software engineering.

So the ladder runs execution understanding → algorithmic coding → repo-level engineering.

What CWM actually showed

The interesting result was *not* "CWM is number one" — it wasn't. It was that **different kinds of world-model data improved the capabilities they should logically improve**: execution traces produced large gains on code-*execution* understanding, and agent–environment trajectories helped repository-level software engineering. That is evidence for the hypothesis

$$\boxed{\text{training on dynamics changes } \textit{what} \text{ the model learns}}$$

rather than world-model data being merely more generic coding tokens. The broader, domain-independent lesson:

$$\boxed{\text{agents may benefit from learning environment dynamics } \textit{before} \text{ RL}}$$

Current agents often learn `observation → action` without being explicitly trained on huge quantities of `observation → action → resulting observation`. CWM suggests that trajectory data is valuable enough to deserve an entire midtraining phase. In compact form the whole decomposition is

$$\text{knowledge} \rightarrow \text{world dynamics} \rightarrow \text{behaviour} \rightarrow \text{reward optimisation.}$$

The long-horizon vision is *internal simulation*: a model that understands dynamics could imagine outcome A, outcome B, outcome C, compare them, and choose — `imagine → evaluate → act`. CWM demonstrates execution prediction but is not yet a fully developed Dreamer-like planner; that remains a research direction, not a solved feature.

Scaling laws and overtraining for inference

Classical Chinchilla-style scaling asks: given fixed *training* compute, what mix of parameters and tokens minimises training loss? But a deployed model also pays *inference* cost:

$$C_{\text{total}} = C_{\text{training}} + N_{\text{inference}}\, C_{\text{inference}}.$$

If a model is served billions of times, a *smaller* model trained on *far more* data becomes economically attractive — you pay more once in training to save inference compute repeatedly. This is why production models are often

deliberately **overtrained relative to the old compute-optimal ratio**. Scaling laws are not wrong; the optimisation target simply moved from "minimise training compute" to "training cost + long-term inference cost + latency + memory footprint."

Connection to a financial world model

This generalises directly to our project. For an economic system the dynamics are

$$P\big(S_{t+1} \mid S_t, A_t, E_t\big),$$

where the state S_t bundles revenue, margin, inventory, capacity, pricing, suppliers/customers, debt, cash, and market share; actions A_t are managerial moves (raise prices, expand capacity, cut production, launch, acquire, issue debt, buy back stock); and E_t are external events (Fed rate changes, oil shocks, tariffs, customer failures, competitor launches, regulation). This is the exact analogue of CWM's $P(\text{next computational state} \mid \text{state}, \text{action})$ — and it is the world-model component that sits behind the RippleSignal of Chapter 1.

But finance differs from Python in ways that must shape the design. Python is approximately deterministic, fully observable, cheap to execute, and easy to verify; the economy is stochastic, partially observed, confounded, and driven by hidden variables. Therefore a financial world model must represent a **distribution over futures**, not one deterministic next state — "20% recession / 60% soft landing / 20% boom," never "the future is definitely a soft landing." Calibration is everything.

The architectural consequence is to keep world modelling *separate* from portfolio optimisation:

```
world model → distribution of future scenarios → fundamental
consequences
→ valuation → portfolio optimiser
```

The world model estimates $p_\phi(\text{future} \mid \text{current information})$; a *separate* component then chooses $\pi_\theta(\text{portfolio action} \mid \text{belief over futures})$ or solves the optimiser of Chapter 1 directly. The separation matters because RL's reward-maximising pressure favours high-reward *actions* and must not be allowed to distort the estimate of **how likely each future actually is**. A useful slogan for the whole progression:

> A normal LLM learns *what comes next*; a world-model-trained LLM also learns *what an action does*; SFT teaches *what good behaviour looks like*; RL teaches *which behaviours produce verified success*.

> **Key takeaway.** CWM's real contribution is a *sequencing* claim, not a leaderboard: teach an agent the dynamics of its environment (a cheap, dedicated midtraining phase on `state → action → state` traces) *before* asking RL to discover how to act. For us the transplant is direct but demands one change — the economic world model must output *calibrated distributions over futures* and be kept strictly upstream of, and independent from, the reward-seeking portfolio layer.

Chapter 3: Engineering Long Contexts

An agent that must reason over a whole repository — or, in our case, over years of filings, transcripts, and a large economic graph — needs a very long context window. Making that practical is a systems problem as much as a modelling one. This chapter collects the machinery: how position is encoded (RoPE) and extended, how variable-length data is batched, how attention cost is reduced (local attention), and how the GPU memory hierarchy forces the designs that make long context affordable (FlashAttention, KV cache, GQA, Ring Attention). None of it is specific to code — it is the plumbing under any long-context agent.

RoPE: where position enters attention

For a hidden state x_p at position p, attention forms $q_p = W_Q x_p$, $k_p = W_K x_p$, $v_p = W_V x_p$. **Rotary Position Embedding (RoPE)** injects position by *rotating* q and k — not v. Each query/key head is split into pairs of dimensions, and a pair (q_1, q_2) is rotated by an angle determined by the token's position:

```
hidden token → Q/K projection → positional rotation → attention
```

The learned W_Q, W_K mainly capture useful content features; RoPE then modifies their output according to position. Because the rotation depends on position, the dot product $q_p \cdot k_{p'}$ ends up depending on the *relative* offset $p - p'$, which is what we want.

RoPE as many clocks. Different dimension-pairs rotate at different speeds — fast clocks, medium clocks, slow clocks. Fast clocks distinguish *nearby* positions; slow clocks distinguish *very distant* ones. Any single clock loops around (aliases), but the *joint* pattern across many clocks stays informative — exactly like reading seconds, minutes, hours, and days together to pin down a moment.

Why long contexts break RoPE

Suppose the model was trained only up to 8K tokens. RoPE can *mathematically* compute the rotation for position 100K, but the network has never *learned* what those positional patterns mean.

> Being mathematically defined at 100K does not mean the model has learned 100K-context behaviour.

So long-context extension needs both a positional scheme that extrapolates sensibly *and* actual long-context training.

Extending context: theta and positional scaling

RoPE uses a family of frequencies roughly like

$$\omega_m = \theta^{-2m/d},$$

for pair index m and head dimension d. Increasing the base θ makes the slower frequencies rotate *even more slowly*, providing more usable low-frequency structure for very long distances — fast frequencies give local detail, slow frequencies give long-distance structure, and a larger θ emphasises the availability of those slow clocks.

A complementary trick is **positional scaling**. A simple intuitive version maps

$$p_{\text{effective}} = \frac{p}{16},$$

without rounding, so position 1 → 0.0625 and position 2 → 0.125 remain distinct (there are no 16-token buckets). The intuition is $131072/16 = 8192$: a 128K-token context is squeezed into a positional phase range close to what the original 8K model already understands. Real implementations use more sophisticated frequency-dependent scaling rather than dividing every position uniformly, but the conceptual goal is the same — **stretch the learned positional coordinate system rather than forcing the network to relearn long-range geometry from scratch.**

Does scaling preserve the old 8K behaviour perfectly? *No.* Changing RoPE changes how q/k are rotated even at short positions, so some additional training is required afterward. But most of the learned machinery — language and code knowledge, the $W_Q/W_K/W_V$ projections, content matching, attention structure — is retained. The model is *not* learning from scratch; only the long-range behaviour is genuinely new.

Batching: padding, bucketing, packing, block-diagonal masks

Training sequences have different lengths, which wastes compute if handled naively.

- **Padding.** A 3-token example next to a 6-token one is padded to equal length; attention masks stop tokens attending to `PAD`, and the loss ignores `PAD` positions. Simple, but wasteful.
- **Bucketing.** Group similar-length sequences (500/550/600 in one batch; 7000/7500/8000 in another) so you still pad, but much less.
- **Packing.** Concatenate several short documents into one physical sequence `[A][B][C]` and use a mask to keep them independent. This uses tokens far more efficiently, and large systems combine bucketing and packing.

Packing needs a **block-diagonal causal mask** so packed documents cannot leak into each other. For `I like cats | Dogs are cool`, "Dogs" must *not* attend to document A's tokens, so the mask is two separate causal triangles on the diagonal:

```
A: ✓
   ✓ ✓
   ✓ ✓ ✓
B:         ✓
           ✓ ✓
           ✓ ✓ ✓
```

Mathematically this makes A and B behave almost like separate sequences even though they share one physical tensor.

Attention cost: full vs local

Full causal attention lets token i attend to all previous tokens $1, \ldots, i$, so cost scales as

$$O(L^2).$$

Local (sliding-window) attention restricts token i to a window $i - W + 1, \ldots, i$, giving

$$O(L\,W),$$

a major saving for large L. A model can *mix* layers — mostly cheap local layers with occasional global layers that communicate across the entire document:

```
local
local
local
global
local
local
local
```

GPU memory and the memory-bound problem

A simplified memory hierarchy, fastest/smallest to slowest/largest:

```
GPU registers / SRAM   very fast, very small
GPU HBM                much larger, slower
CPU RAM                larger, slow link to the GPU
SSD                    slower again
```

The bottleneck is often **HBM traffic** — and it is not that "writes are bad but reads are fine"; both reads and writes consume bandwidth. A kernel is **memory-bound** when it waits mostly on data movement and **compute-bound** when it

waits mostly on arithmetic. The deciding quantity is *arithmetic intensity*,

$$\text{arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

High intensity means lots of useful math per byte loaded. Because modern GPUs are extraordinarily fast at matrix multiplication, memory movement is increasingly the limiting factor.

FlashAttention

Naive attention is memory-bound because it materialises the huge $L \times L$ matrices in HBM and round-trips them:

```
QKᵀ → write L×L scores to HBM
read scores → softmax → write L×L probs to HBM
read probs → multiply by V
```

FlashAttention avoids ever materialising the full attention matrix. It streams in blocks:

```
load a Q block; load a K/V block
compute the score block on-chip
update an online softmax; accumulate weighted V immediately
discard the temporary score block; move to the next block
```

Temporaries stay in fast on-chip memory. You still read $Q/K/V$, and for *full* attention the arithmetic is still fundamentally $O(L^2)$ pair interactions — the win is eliminating the repeated HBM reads/writes of the enormous score and probability matrices, which makes it dramatically faster and more memory-efficient.

FlashAttention and local attention solve *different* problems and compose: FlashAttention makes the required attention computation memory-efficient, while local attention *reduces how many interactions you need at all*. Together — fewer interactions, and less memory traffic per interaction — they are especially powerful for long context.

KV cache and GQA

During autoregressive generation the previous tokens' K/V do not change, so recomputing them each step is wasteful. The **KV cache** stores $K_1, V_1, \ldots, K_n, V_n$; to generate the next token you compute only its new q, k, v, attend against the cache, and append the new k, v. For active high-performance inference the cache normally lives in **GPU HBM** if it fits; CPU offload is possible but slower.

Local attention helps the cache: a local layer with an 8K window only needs its most recent 8K entries, so older entries can be *permanently discarded* and its cache behaves like a ring buffer. A global layer is different — it may still need every previous position, so its cache keeps growing with context length.

Grouped Query Attention (GQA) shrinks the cache by using more query heads than key/value heads — e.g. 48 query heads sharing 8 K/V heads (6 queries per K/V head). The shared K/V heads are genuinely shared representations, not duplicated copies. This matters enormously at long context because KV-cache memory scales as

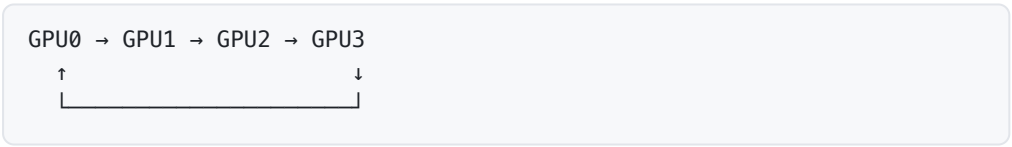
$$\text{cache} \;\propto\; (\text{seq length}) \times (\text{layers}) \times (\text{KV heads}) \times (\text{head dim}),$$

so cutting the number of K/V heads can save many gigabytes.

Note that KV cache and FlashAttention answer different questions — KV cache: "how do I avoid recomputing old K/V each step?"; FlashAttention: "given the $Q/K/V$ I must process, how do I attend without wasteful memory traffic?" They complement each other: old K/V sit in HBM, and a FlashAttention-style kernel reads them in blocks to produce the result.

Ring Attention and the Large World Model

Ring Attention addresses a different limit: when the *sequence itself* is too large to live on one GPU. Split 1M tokens across four GPUs (250K each); each GPU keeps its own *query* block, and the K/V blocks circulate around a ring:



At each step a GPU computes its local queries against whichever K/V block is passing through, then forwards that block on. After a full rotation every query block has interacted with every required K/V block, and an online softmax lets each GPU accumulate its result incrementally. The distinction from FlashAttention:

- **FlashAttention** tiles attention *inside one GPU's* memory hierarchy.
- **Ring Attention** distributes attention *across multiple GPUs* by splitting the sequence dimension.

They compose — each GPU in a ring can still run FlashAttention-like kernels internally. But Ring Attention does *not* remove the quadratic cost: exact full attention is still $\approx O(L^2)$ arithmetic in total. What it solves is per-GPU memory limits, sequence distribution, and communication/computation overlap. For truly enormous contexts you additionally attack the $O(L^2)$ term with local or sparse attention, retrieval, recurrence, or hierarchical memory.

A concrete example of Ring Attention at scale is the **Large World Model (LWM)** — a 2024 UC Berkeley project (Hao Liu, Wilson Yan, Matei Zaharia, Pieter Abbeel and collaborators), *not* Meta's CWM. LWM explored autoregressive models with extremely long text/video contexts, progressively extended toward roughly **1 million tokens**, using Blockwise RingAttention to distribute those huge sequences across GPUs. Its topic is long-context *multimodal* modelling rather than computational world models, but it is the canonical demonstration of Ring Attention at million-token scale (arXiv:2402.08268).

> **Key takeaway.** Long context is bought with a stack of complementary tricks, each aimed at a different bottleneck: RoPE (plus θ increase / positional scaling *and* retraining) makes positions *meaningful* far past the training length; local attention cuts the number of interactions from $O(L^2)$ toward $O(LW)$; FlashAttention cuts the *memory traffic* per interaction; KV cache and GQA cut *inference* memory; and Ring Attention spreads a single giant sequence across GPUs. They stack because they attack orthogonal costs — position, interaction count, memory traffic, cache size, and per-device capacity.

Chapter 4: Neural Networks, Universal Approximation, Depth, and Function Approximation

The harness (Chapter 1) and the code/finance world models (Chapters 2–3) all rest on one primitive: a neural network approximating a function we cannot write down. This chapter builds that primitive from the ground up in the cleanest possible setting — a one-hidden-layer ReLU network approximating a continuous function on an interval — and then shows exactly *why* it works, *how* depth buys efficiency, and where neural nets sit among the other universal approximators (polynomials, Fourier series, RBFs). The spine of the whole story is a single template: approximate a hard function as a weighted sum of simpler basis functions, $f(x) \approx \sum_i a_i \phi_i(x)$. Everything below is a variation on what the ϕ_i are and how they combine.

The central idea

A neural network can approximate complicated functions by combining many simpler nonlinear functions.

A one-hidden-layer network has the form

$$ \hat f(x)

\sum_{j=1}^{m} a_j,\sigma(w_j^\top x+b_j). $$

You can think of each hidden neuron as creating one learned nonlinear feature

$$ \phi_j(x)

\sigma(w_j^\top x+b_j), $$

and then the output layer combines these features:

$$ \hat f(x)

\sum_j a_j\phi_j(x). $$

This resembles many other approximation methods:

$$f(x) \approx \sum_i a_i \phi_i(x).$$

The difference is that in a neural network, the functions ϕ_i are themselves learned through w_i and b_i. That is one of the main reasons neural networks are so flexible.

Why do we need nonlinear activation functions?

Suppose there were no activation function. Then two linear layers would give

$$W_2(W_1 x + b_1) + b_2.$$

Expanding,

$$W_2 W_1 x + W_2 b_1 + b_2.$$

Define

$$A = W_2 W_1, \qquad c = W_2 b_1 + b_2.$$

Then the whole network is simply

$$Ax + c.$$

So even if the hidden layer had a million neurons, a stack of purely linear layers would still be just one linear transformation. Therefore the nonlinearity is essential:

$$\boxed{\text{linear layers alone cannot represent nonlinear functions}}$$

while

$$\boxed{\text{linear layers} + \text{nonlinear activations can.}}$$

What a ReLU neuron does

The ReLU function is

$$ \operatorname{ReLU}(z)

\max(0,z). $$

Equivalently,

$$ \operatorname{ReLU}(z)

\begin{cases} 0, & z<0,\[4pt] z, & z\ge 0. \end{cases} $$

A neuron computes

$$ h(x)

\operatorname{ReLU}(w^\top x+b). $$

In one dimension,

$$ h(x)

\operatorname{ReLU}(wx+b). $$

The point where the behavior changes is where

$$wx + b = 0.$$

So the breakpoint is

$$x = -\frac{b}{w}.$$

Before that point, the neuron may output zero. After that point, it behaves linearly. That is why a ReLU neuron is often called a **hinge**.

In higher dimensions, the breakpoint becomes a hyperplane

If $x \in \mathbb{R}^d$, then

$$w^\top x + b = 0$$

defines a hyperplane. In two dimensions, this is a line. In three dimensions, this is a plane. In higher dimensions, it is the natural generalization.

So each ReLU neuron divides input space into two sides:

$$w^\top x + b < 0$$

and

$$w^\top x + b > 0.$$

On one side the neuron is inactive. On the other side it contributes a linear function.

Why a ReLU network is piecewise linear

Suppose we have many ReLU neurons. Within some region of input space, assume we know exactly which neurons are active and which are inactive.

For an active neuron,

$$ \operatorname{ReLU}(w^\top x+b)

w^\top x+b. $$

For an inactive neuron,

$$ \operatorname{ReLU}(w^\top x+b)

0.

$$

So once the activation pattern is fixed, the entire network reduces to an affine function:

$$Ax + c.$$

Therefore,

$$\boxed{\text{a ReLU network is globally nonlinear but locally linear.}}$$

The input space is divided into many **linear regions**, and inside each region the network behaves like a linear function.

How ReLUs create a triangular bump

A single ReLU is an unbounded ramp. But several ReLUs can cancel each other and create something localized. Consider

$$ h(x)

\operatorname{ReLU}(x)

2\operatorname{ReLU}(x-1) + \operatorname{ReLU}(x-2). $$

Compute it region by region. For $x < 0$, all terms are zero, so $h(x) = 0$. For $0 \leq x < 1$, only the first ReLU is active: $h(x) = x$. For $1 \leq x < 2$, the first two are active:

$$ h(x)

x-2(x-1)

2-x. $$

For $x \geq 2$, all three are active:

$$ h(x)

x-2(x-1)+(x-2)

0.

$$

Therefore,

$$ h(x)

\begin{cases} 0, & x<0,\[4pt] x, & 0\le x<1,\[4pt] 2-x, & 1\le x<2,\[4pt] 0, & x\ge2. \end{cases} $$

This is exactly a triangle. So:

several ReLU ramps can combine into a localized bump.

This is one intuitive route toward understanding universal approximation.

What are x and x_i?

Suppose we want to approximate a continuous function such as $f(x) = x^2$ on $[0, 4]$. Here, x is the variable input. It can be any real number in the interval.

Now choose specific grid points

$$x_0 = 0, \quad x_1 = 1, \quad x_2 = 2, \quad x_3 = 3, \quad x_4 = 4.$$

These x_i's are fixed sample locations. At those points we know the true values $f(x_i)$. For $f(x) = x^2$,

$$f(0) = 0, \quad f(1) = 1, \quad f(2) = 4, \quad f(3) = 9, \quad f(4) = 16.$$

Why multiply bumps by $f(x_i)$?

Suppose $h_i(x)$ is a triangular bump centered at x_i, with $h_i(x_i) = 1$. If we want the approximation to have height $f(x_i)$ at x_i, simply multiply: $f(x_i)h_i(x)$. Then

$$ f(x_i)h_i(x_i)

f(x_i). $$

So the bump's shape determines where it contributes, while the coefficient $f(x_i)$ determines its height. If we have many bumps, we sum them:

$$ \hat f(x)

\sum_i f(x_i)h_i(x). $$

Each bump contributes mostly near its own grid point. This is the same broad idea used in many approximation schemes:

$$\boxed{\text{local basis functions} + \text{appropriate coefficients} \to \text{global approximati}}$$

Piecewise-linear interpolation

A cleaner proof for ReLU networks uses line segments directly. Take grid points

$$x_0 < x_1 < \cdots < x_n.$$

At each grid point, evaluate the real function:

$$f(x_0), f(x_1), \ldots, f(x_n).$$

Then connect $(x_i, f(x_i))$ to $(x_{i+1}, f(x_{i+1}))$ with a straight line. The resulting function is called the **piecewise-linear interpolant** $p(x)$. It satisfies

$$p(x_i) = f(x_i)$$

at every grid point. Between grid points it is a straight-line approximation to the real function.

Why does making the grid finer improve the approximation?

This is where uniform continuity enters. Suppose $f : [a, b] \to \mathbb{R}$ is continuous. Because $[a, b]$ is compact, f is uniformly continuous. That means: for every $\varepsilon > 0$, there exists a single $\delta > 0$ such that for every $x, y \in [a, b]$,

$$|x - y| < \delta$$

implies

$$|f(x) - f(y)| < \varepsilon.$$

The important part is that the same δ works everywhere. Now choose the grid fine enough that $x_{i+1} - x_i < \delta$. Take any point $x \in [x_i, x_{i+1}]$. Because x is within δ of both endpoints,

$$|f(x) - f(x_i)| < \varepsilon$$

and

$$|f(x) - f(x_{i+1})| < \varepsilon.$$

The line $p(x)$ between the endpoint values can be written as

$$ p(x)

(1-\lambda)f(x_i) + \lambda f(x_{i+1}), $$

where $0 \leq \lambda \leq 1$. Then

$$ f(x)-p(x)

(1-\lambda)\bigl(f(x)-f(x_i)\bigr) + \lambda\bigl(f(x)-f(x_{i+1})\bigr). $$

Taking absolute values,

$$|f(x) - p(x)| \leq (1-\lambda)|f(x) - f(x_i)| + \lambda|f(x) - f(x_{i+1})|.$$

Since both terms are less than ε,

$$|f(x) - p(x)| < \varepsilon.$$

Thus:

a sufficiently fine piecewise-linear interpolation approximates any co

How ReLU represents the whole piecewise-linear function

Now we need to show that the piecewise-linear function $p(x)$ can itself be represented by ReLUs. Suppose the slopes of the different segments are $m_0, m_1, m_2, \ldots$. The general representation is

$$ \boxed{ p(x)

a+bx+ \sum_{i=1}^{n} c_i\operatorname{ReLU}(x-x_i). } $$

Here a sets the initial vertical offset, b sets the initial slope, and each ReLU changes the slope at one breakpoint. The initial slope is $b = m_0$. At breakpoint x_i, choose $c_i = m_i - m_{i-1}$.

Why? Before x_i, $\mathrm{ReLU}(x - x_i) = 0$. After x_i, $\mathrm{ReLU}(x - x_i) = x - x_i$, whose slope is 1. Multiplying by c_i changes the slope by c_i. Therefore, if the old slope was m_{i-1}, the new slope becomes $m_{i-1} + c_i$. Choosing $c_i = m_i - m_{i-1}$ gives $m_{i-1} + c_i = m_i$. So each ReLU literally implements a desired **change in slope**.

Full example: approximating z^2

Consider $f(z) = z^2$ on $[-2, 2]$. Choose grid points $-2, -1, 0, 1, 2$. The true values are $4, 1, 0, 1, 4$. The slopes between consecutive points are

$$ m_0

\frac{1-4}{-1-(-2)}

-3, $$

$$ m_1

\frac{0-1}{0-(-1)}

-1, $$

$$ m_2

\frac{1-0}{1-0}

1, $$

$$ m_3

\frac{4-1}{2-1}

3.

$$

So the slope changes are

$$c_1 = m_1 - m_0 = 2, \qquad c_2 = m_2 - m_1 = 2, \qquad c_3 = m_3 - m_2 = 2.$$

The initial slope is $b = -3$. To match $p(-2) = 4$, we choose $a = -2$.
Therefore,

$$ \boxed{ S(z)

-2 -3z + 2\operatorname{ReLU}(z+1) + 2\operatorname{ReLU}(z) + 2\operatorname{ReLU}(z-1). } $$

This exactly represents the straight-line interpolation through $(-2, 4)$, $(-1, 1)$, $(0, 0)$, $(1, 1)$, $(2, 4)$. So $S(z) \approx z^2$. A finer grid gives a better approximation.

Representing the linear term using only ReLUs

A useful identity is

$$ \boxed{ z

\operatorname{ReLU}(z)

\operatorname{ReLU}(-z). } $$

To verify: if $z > 0$, $\mathrm{ReLU}(z) = z$ and $\mathrm{ReLU}(-z) = 0$. If $z < 0$, $\mathrm{ReLU}(z) = 0$ and $\mathrm{ReLU}(-z) = -z$. In either case, $\mathrm{ReLU}(z) - \mathrm{ReLU}(-z) = z$. Therefore,

$$ -3z

-3\operatorname{ReLU}(z) + 3\operatorname{ReLU}(-z). $$

So the square approximation can be written using only ReLU hidden units and a linear output layer.

The 1D universal approximation proof

We can now summarize the proof. Suppose $f : [a, b] \to \mathbb{R}$ is continuous. Given any desired error $\varepsilon > 0$, uniform continuity lets us choose sufficiently fine grid points $x_0, \ldots, x_n$ such that the piecewise-linear interpolation $p(x)$ satisfies

$$\sup_{x \in [a,b]} |f(x) - p(x)| < \varepsilon.$$

But every continuous piecewise-linear $p(x)$ can be represented exactly as

$$ p(x)

a+bx+ \sum_i c_i\operatorname{ReLU}(x-x_i). $$

Therefore a one-hidden-layer ReLU network can approximate f to arbitrary accuracy. That gives the core 1D universal approximation argument:

$$\boxed{\text{continuous function} \to \text{fine piecewise-linear approximation} \to \text{ReLU}}$$

What the general universal approximation theorem says

A common form is: for continuous $f : K \to \mathbb{R}$, where $K \subset \mathbb{R}^d$ is compact, and for every $\varepsilon > 0$, there exists a sufficiently wide neural network such that

$$ \boxed{ \sup_{x\in K} \left| f(x)

\sum_{j=1} ^ {m} a_j\sigma(w_j ^ \top x+b_j) \right| < \varepsilon. } $$

The supremum means the worst-case error over the entire domain. So the theorem says: give me any continuous target function and any error tolerance, and there exists a sufficiently large one-hidden-layer network whose error is below that tolerance everywhere on the compact domain.

But this is only an **existence result**. It does not tell us:

- how wide the network must be,
- whether SGD will find the right parameters,
- whether training is easy,
- whether the learned model generalizes.

Width versus depth

Width and depth provide different kinds of expressive power. Width gives more nonlinear features in parallel. In one dimension, more ReLUs mean more possible breakpoints. So roughly,

$$\boxed{\text{width} \Rightarrow \text{more pieces at the same level.}}$$

Depth lets us compose functions:

$$ f(x)

f_L(f_{L-1}(\cdots f_1(x))). $$

The key advantage is that later nonlinearities operate on a representation that has already been transformed by earlier nonlinearities. This can make the number of effective regions grow multiplicatively.

Why depth can create exponentially many pieces

Consider the triangle function

$$ T(x)

\begin{cases} 2x, &0\le x\le\frac12, \[6pt] 2-2x, &\frac12<x\le1. \end{cases}
$$

It has two linear pieces. Now compose it with itself: $T(T(x))$. The first T maps $[0, \frac{1}{2}]$ from 0 to 1, and maps $[\frac{1}{2}, 1]$ from 1 back to 0. Now the second T has a breakpoint when its input equals $\frac{1}{2}$. So solve $T(x) = \frac{1}{2}$. On the first branch, $2x = \frac{1}{2}$, so $x = \frac{1}{4}$. On the second branch, $2 - 2x = \frac{1}{2}$, so $x = \frac{3}{4}$.

Thus one breakpoint in the second layer becomes two breakpoints in the original input space. The result has four linear pieces. Compose again: $T(T(T(x)))$. Now the folded representation crosses the threshold multiple times, producing eight pieces. Thus, after L compositions,

$$\boxed{T^{\circ L}(x) \text{ can have } 2^L \text{ linear pieces.}}$$

This is why depth can create exponentially more regions than a shallow network with a comparable number of units. The key idea is:

$$\boxed{\text{later layers reuse their nonlinear boundaries across all regions create}}$$

Why a shallow network may need exponentially more width

A one-hidden-layer ReLU network in 1D has the form

$$ f(x)

a+bx+ \sum_{j=1}^{m} c_j\operatorname{ReLU}(x-t_j). $$

Each hidden unit contributes one breakpoint t_j. So the number of breakpoints grows roughly linearly with m. To reproduce a function with about 2^L alternating linear pieces, the shallow network may need on the order of 2^L hidden units. A deep network may create similar complexity with only L repeated stages. So:

depth can be exponentially more efficient for certain compositional fu

Multiplication using a ReLU network

A useful identity is

$$ \boxed{ xy

\frac14 \left[(x+y) ^ 2-(x-y) ^ 2 \right]. } $$

A linear layer can compute exactly $z_1 = x + y$ and $z_2 = x - y$. Then use the ReLU square approximator S:

$$S(z_1) \approx z_1^2, \qquad S(z_2) \approx z_2^2.$$

Finally compute

$$ \hat m(x,y)

\frac14 \left[S(z_1)-S(z_2) \right]. $$

Therefore,

$$\boxed{\hat{m}(x, y) \approx xy.}$$

This gives a concrete example of hierarchical computation:

$$(x, y) \to (x + y, x - y) \to \left((x + y)^2, (x - y)^2\right) \to xy.$$

The addition and subtraction are exact linear operations. The nonlinear square is approximated by ReLUs.

Polynomial approximation

Neural networks are not the only universal approximators. The Weierstrass approximation theorem says that if $f : [a, b] \to \mathbb{R}$ is continuous, then for every $\varepsilon > 0$ there exists a polynomial

$$ p(x)

a_0+a_1x+\cdots+a_nx^n $$

such that

$$\sup_{x\in[a,b]} |f(x) - p(x)| < \varepsilon.$$

So polynomial basis functions are $1, x, x^2, x^3, \ldots$. This is another example of the general principle $f(x) \approx \sum_i a_i \phi_i(x)$.

Fourier series

Fourier methods use sine and cosine functions:

$$f(x) \approx \frac{a_0}{2} + \sum_{k=1}^{N} \left[a_k \cos(kx) + b_k \sin(kx) \right].$$

The basis functions are fixed: $\sin(kx)$, $\cos(kx)$. The interesting fact is that individual sine waves are global, but their sums can create highly localized structure through cancellation and reinforcement. So:

$$\boxed{\text{simple global waves} \rightarrow \text{complex functions through interference.}}$$

Fourier series can approximate broad classes of periodic functions, including continuous periodic functions under appropriate convergence results.

RBF networks

RBF stands for radial basis function. A common RBF unit is a Gaussian bump:

$$ \phi_i(x)

\exp\left(-\frac{|x-c_i| ^2}{2\sigma_i ^2} \right). $$

Here c_i is the center of the bump, and σ_i controls its width. An RBF network is

$$ \hat f(x)

\sum_{i=1} ^ {m} a_i\phi_i(x). $$

So one neuron already gives a localized bump. This makes the approximation intuition very direct. However, in high dimensions, local bumps may require enormous numbers of centers. If we need k locations along each of d dimensions, a naïve grid scales like k^d. This is the curse of dimensionality. Deep neural networks can sometimes avoid this by reusing learned structure across layers.

Dead ReLUs

The derivative of ReLU is

$$ \frac{d}{dz}\operatorname{ReLU}(z)

\begin{cases} 0, & z<0,\[4pt] 1, & z>0. \end{cases} $$

If a neuron has $z < 0$ for one input, it is simply inactive for that input. That does not mean it is permanently dead. A neuron becomes effectively dead when $w^\top x + b < 0$ for essentially all training examples for a long time. Then its own incoming weights may receive almost no gradient. However, in a deep network, earlier layers may continue changing because of other active paths. So the neuron's input can change later and possibly reactivate it. Thus:

$$\boxed{\text{inactive for one example} \neq \text{permanently dead neuron.}}$$

Why sums suggest Gaussian behavior

Suppose $X = X_1 + \cdots + X_n$. Under suitable independence and finite-variance assumptions, the central limit theorem says that the normalized sum approaches a Gaussian distribution. This is one reason pre-activations such as $z = \sum_i w_i x_i$ are often approximately Gaussian at initialization. So:

$$\boxed{\text{many additive contributions} \to \text{Gaussian-like behavior.}}$$

Why products suggest log-normal behavior

Suppose $Y = A_1 A_2 \cdots A_n$ with positive A_i. Take logarithms:

$$ \log Y

\sum_i \log A_i. $$

If the terms $\log A_i$ satisfy CLT-like conditions, then $\log Y$ can become approximately Gaussian:

$$\log Y \sim \mathcal{N}(\mu, \sigma^2).$$

Then by definition, Y is approximately log-normal:

$$Y \sim \text{LogNormal}(\mu, \sigma^2).$$

So the conceptual chain is

$$\boxed{\text{product} \xrightarrow{\log} \text{sum} \xrightarrow{\text{CLT}} \text{Gaussian in log-space}}$$

which implies

$$\boxed{\text{log-normal in ordinary space.}}$$

A log-normal distribution is positive, asymmetric, and has a long right tail. This is useful intuition for multiplicative processes such as products of Jacobians in deep networks, but it does not imply gradients are exactly log-normal.

Gradient norms versus gradient histograms

Suppose a layer has gradient vector $g = (g_1, \ldots, g_n)$. The gradient norm is

$$ |g|_2

\sqrt{\sum_i g_i^2}. $$

This tells us how large the layer's gradient is overall. Tracking $\|g_t\|$ over training can reveal exploding gradients, vanishing gradients, sudden optimization instability, or layers receiving almost no learning signal. But the norm loses information about how the gradient is distributed across coordinates. For example,

$$g_1 = (1, 1, 1, 1) \qquad \text{and} \qquad g_2 = (2, 0, 0, 0)$$

both satisfy $\|g_1\|_2 = \|g_2\|_2 = 2$. Yet they are very different. The first spreads gradient evenly. The second concentrates everything in one coordinate. That is why histograms are also useful.

What gradient histograms can reveal

A histogram of individual gradient values can reveal:

- **Outliers** — most gradients are small, but a few are extremely large.
- **Sparsity** — a huge fraction of values lie near zero.
- **Bimodality** — two distinct peaks, suggesting two populations of parameters or gradients.
- **Asymmetry** — positive and negative gradients behave differently.
- **Heavy tails** — large values occur more frequently than expected under a Gaussian.
- **Clipping** — values pile up at the clipping thresholds.
- **Quantization effects** — continuous values collapse onto discrete levels.

So:

$$ \boxed{ \text{norm}}

\text{overall magnitude} } \qquad \boxed{ \text{histogram}

\text{structure of individual coordinates}. } $$

Both are useful.

Topology interlude: open, closed, and bounded sets

The universal approximation theorem is stated on a *compact* domain, so it is worth pinning down the analysis vocabulary it relies on.

A set S is **open** if every point has a small neighborhood completely inside the set. Formally, $\forall x \in S$, there exists $r > 0$ such that $B(x, r) \subseteq S$. For example, $(0, 1)$ is open — no point includes the boundary 0 or 1.

A set S is **closed** if it contains all its limit points. Equivalently, if $x_n \in S$ and $x_n \to x$, then $x \in S$. For example, $[0, 1]$ is closed: the sequence $\frac{1}{n}$ lies inside $[0, 1]$ and converges to 0, which is also inside the set. In contrast, $(0, 1)$ is not closed because $\frac{1}{n} \to 0$ but $0 \notin (0, 1)$.

A set is **bounded** if it does not extend arbitrarily far. Formally, there exists $M < \infty$ such that $\|x\| \leq M$ for every point in the set. For example, $[0, 1]$ is bounded, but $[0, \infty)$ is not.

Cauchy sequences and completeness

A sequence $x_1, x_2, \ldots$ is **Cauchy** if its terms eventually become arbitrarily close to one another. Formally, $\forall \varepsilon > 0$, there exists N such that whenever $m, n > N$, we have $d(x_m, x_n) < \varepsilon$.

A metric space is **complete** if every Cauchy sequence converges to a point inside that space. The real numbers are complete:

$$\boxed{\mathbb{R} \text{ is complete.}}$$

The rationals are not. A sequence of rational approximations can converge to $\sqrt{2}$, but $\sqrt{2} \notin \mathbb{Q}$. So $\mathbb{Q}$ contains a "hole." A useful intuition is:

$$ \boxed{ \text{complete}

\text{no missing limits for Cauchy sequences}. } $$

Compactness

The formal definition is: a set is compact if every open cover has a finite subcover. That definition is abstract. In Euclidean space, however, the Heine–Borel theorem gives a simpler characterization:

$$\boxed{K \subset \mathbb{R}^d \text{ is compact } \iff K \text{ is closed and bounded.}}$$

So $[0,1]$ is compact. But $(0,1)$ is not compact because it is not closed. And $[0,\infty)$ is not compact because it is unbounded. Another useful equivalent characterization in $\mathbb{R}^d$ is: every sequence in a compact set has a convergent subsequence whose limit remains inside the set. Intuitively:

$$\boxed{\text{bounded prevents escape to infinity}} \qquad \boxed{\text{closed prevents limits from}}$$

Compact versus complete

These are related but different. Complete means: every Cauchy sequence converges inside the space. Compact means something stronger. In metric spaces,

$$\boxed{\text{compact} \Rightarrow \text{complete and bounded.}}$$

But complete and bounded does not imply compact in every possible metric space. In finite-dimensional Euclidean space $\mathbb{R}^d$, things are especially nice because closed and bounded is equivalent to compact.

The final picture

The main approximation theme is

$$\boxed{f(x) \approx \sum_i a_i \phi_i(x).}$$

Different methods use different functions ϕ_i:

$$\phi_i(x) = x^i \quad (\text{polynomials}),$$

$$\phi_i(x) = \sin(ix), \cos(ix) \quad (\text{Fourier}),$$

$$ \phi_i(x)

\exp\left(-\frac{|x-c_i|^2}{2\sigma_i^2} \right) \quad\text{(RBF)}, $$

$$ \phi_i(x)

\operatorname{ReLU}(w_i^\top x+b_i) \quad\text{(ReLU networks)}. $$

The core neural-network ideas to remember are:

$$ \boxed{ \text{width}}

\text{more nonlinear features in parallel} } \qquad \boxed{ \text{depth}}

\text{composition and reuse of nonlinear features}. } $$

A shallow network can approximate any continuous function if it is wide enough. A deep network can often represent structured functions much more efficiently because each new layer operates on a nonlinear representation created by earlier layers.

> **Key takeaway.** Universal approximation and depth answer two different questions. Universal approximation is an *existence* result: any continuous function on a compact set is the limit of $\sum_j a_j \, \mathrm{ReLU}(w_j^\top x + b_j)$ — you build it by interpolating f on a fine grid (uniform continuity guarantees the error shrinks) and realizing each slope change with one ReLU. It says *what is possible*, but is silent on width, on whether SGD finds the weights, and on generalization. Depth answers *how efficiently*: composing nonlinearities folds the input space so linear regions multiply — 2^L pieces from L layers versus $\sim 2^L$ hidden units in one wide layer — so compositional functions (like the finance world model's stacked mechanisms) are exponentially cheaper deep than wide. Neural nets are just one choice of basis ϕ_i in $f \approx \sum_i a_i \phi_i$; what makes them special is that the ϕ_i are *learned*, and that depth lets them be *reused*.

Chapter 5: The Agent Harness — the Machine Around the Model

Chapters 1–4 were about the *model* and the world it approximates: the hedge-fund harness that feeds it (Ch. 1), the training that gives it dynamics (Ch. 2), the context engineering that lets it read long inputs (Ch. 3), and the function-approximation theory underneath it all (Ch. 4). This chapter is about the other half of every real system: a model that can call tools is *not yet an agent*. The **harness** is the software system that turns raw model calls into a reliable autonomous worker — the loop that keeps it going, the tools it acts through, the context it reasons over, the sandbox that keeps it safe, and the tests that tell you it actually worked. This is the *general* version — the machine behind Claude Code, OpenCode, SWE-agent, OpenHands and Aider — component by component, each defined before it is used. It is the direct sibling of Chapter 1: that was the harness for a *finance signal*; this is the harness for a *coding/agentic worker*, and both make the same argument — the discipline around the model, not the model alone, is where the trust lives.

The harness is the machine, not the model

A chat model answers one prompt. An agent pursues a goal across dozens of steps, touching a real environment. Everything between "one model call" and "a trustworthy autonomous worker" is the harness.

> An **agent harness** (also called the *scaffold* or the *agent–computer interface*) is everything around the model weights that lets the model *act*: the loop that calls the model and runs what it asks; the tools it acts through; the context assembled into each call; the sandbox and permissions that bound its power; the verification that feeds real results back; and the memory, tracing and evaluation that make runs durable and measurable.

The analogy that makes it stick: the model is a brilliant new hire who can reason about anything but has *no hands, no memory between meetings, and no idea what's safe to touch*. The harness gives them hands (tools), a desk with only the right papers on it (context), a rule about which drawers need a manager's sign-off (permissions), a way to check their own work before calling it done (verification), and a notebook that survives to tomorrow (memory). Same hire — but now the work is bankable.

The single most important empirical claim in this whole area is that *the interface beats the model*. SWE-agent (Princeton, NeurIPS 2024) found that redesigning the interface a **fixed** model acts through moves its success rate more than most model upgrades do. The corollary is a formula worth internalising:

$$\text{capability you measure} \;=\; \text{harness} \times \text{model}, \quad \text{never the model alone}$$

A weak harness makes a strong model look dumb; a strong harness is where a small, disciplined team can actually win.

Workflow or agent? Autonomy is a cost knob

Before building a loop, decide whether you even need one. Anthropic's *Building Effective Agents* draws the line: a **workflow** is LLM calls wired together on *predefined* code paths — you know the steps in advance. An **agent** is an LLM that *dynamically* directs its own tool use in a loop, for problems where "it's difficult or impossible to predict the required number of steps." Autonomy is a **cost-and-risk knob, not a badge**: use the simplest thing that works, and only climb to a full loop when the task is genuinely open-ended.

The classic way an agent project dies is to wire up the exciting part (model + a big pile of tools + a loop) first, watch a demo succeed once, and only later discover it loops forever on the second task, silently "believes" it fixed a bug it never re-tested, floods its own context until it forgets the goal, or runs `rm -rf` because nothing gated the shell. **Every one of those is a harness bug, not a model bug** — and each maps to one of the eight components below whose whole job is to prevent it.

Eight cross-cutting components wrap every single model call. At run time a call flows $1 \to 8$ and loops; but you *build* in a different order (see the last section). Hold the shape in your head:

$$\text{loop} \to \text{tools} \to \text{context} \to (\text{fan out}) \to \text{gated by permissions} \to \text{verif}$$

Component 1 — the agent loop

The beating heart is almost embarrassingly simple: a `while` loop. Its whole difficulty is in the **stop condition** and in anchoring every turn to reality.

> The **agent loop** is the controller that (1) calls the model, (2) executes whatever tools the model requested, (3) appends the results to the conversation, and (4) calls the model again — repeating until the model returns an answer with no tool calls, or a budget/stop condition fires.

This is the **ReAct** pattern (Yao et al., 2022): interleave *reasoning* ("thought"), *acting* (a tool call) and *observing* (the result) in one trajectory. Claude Code frames the same three beats as **gather context → take action → verify results**, chained across dozens of steps.

```
# The entire agent, minus the cleverness
messages = [system_prompt, user_goal]
while True:
    reply = model(messages, tools)          # think
    if not reply.tool_calls:                 # stop condition: model
        is done
        return reply.text
    for call in reply.tool_calls:            # act
        result = run(call)                   # ← real environment,
        ground truth
        messages.append(observation(result)) # observe
    if steps > MAX or budget_exceeded():     # the OTHER stop
        condition
        return escalate_to_human()
```

The one rule that makes it work: every turn, the agent must gain *ground truth from the environment* — the actual tool result, the actual test output — not its own guess about what happened. The loop is a **feedback controller**: it only converges if it can see the error between "what I intended" and "what actually happened." An agent that acts and then *assumes* success has **opened the loop**, and open loops drift.

- **Good:** "Fix the failing tests" → run the suite → read the errors → search the source → edit → re-run the suite → stop when green. A real success condition (tests pass) *and* a budget cap with human escalation if it stalls.
- **Bad:** a hardcoded 5-step pipeline for an open-ended task (too rigid); a loop with no environment feedback where the model declares victory without re-running anything (never converges); or no cap at all (runaway cost, compounding errors).

> Give the loop **two** stop conditions: a *success* condition read from the environment (tests green, goal state reached) and a *safety* condition (max steps / token budget / wall-clock) that escalates to a human instead of spinning. And prefer a workflow over a loop whenever you can actually predict the steps.

Component 2 — the tool interface (the ACI)

If you get one component right, make it this one. The evidence that interface design beats model size is the strongest single result in the field.

> The **ACI** (agent–computer interface; SWE-agent, Princeton) is the purpose-built set of commands and feedback a model acts through — the agent's equivalent of a human's GUI. The thesis: **LLM agents are a new category of end user**, and just as a good UI makes a person more capable without making them smarter, a good ACI makes a *fixed* model dramatically more capable.

The receipt everyone cites is SWE-agent's ablation on SWE-bench Lite, **same model throughout**: the purpose-built edit tool alone was worth a **7.7-point** swing in resolve rate — larger than many model upgrades — with the weights held fixed. Aider's parallel result: switching GPT-4 Turbo to a rigid unified-diff edit format cut "lazy coding" roughly $3\times$ and lifted its benchmark from **20% → 61%**.

The five rules of a good tool:

1. **Few, high-impact, consolidated** — not thin API wrappers. One `search_contacts` beats making the agent chain `list_users → list_events → filter` and burn context doing your join for you.
2. **Namespace them** — `asana_search`, `gdrive_search` — so related tools cluster and the model picks the right one.
3. **Return meaning, not opaque IDs** — give back `"Jane Smith / #eng-team"`, not a raw UUID. Anthropic found resolving UUIDs to human-readable names "significantly improves precision."
4. **Make errors actionable** — `"File not found — did you mean src/app.py? Use an absolute path."`, not a bare stack trace. *The error message is a prompt back to the model.*
5. **Budget the output** — Claude Code caps a single tool response at ~25,000 tokens by default; a huge `grep` is truncated with a note, not allowed to blow the window.

The governing principle is **poka-yoke** (Japanese manufacturing: "mistake-proofing"). If the agent keeps passing relative paths that break, don't add a warning to the docstring — *change the tool signature to require an absolute path* so the wrong call can't even be expressed. The best tool interface makes the correct action the only representable one. Anthropic reports that small refinements to *tool descriptions alone* pushed Sonnet to state-of-the-art on SWE-bench Verified — this is the highest-leverage tuning surface in the harness.

Component 3 — context management

The context window is not free storage you fill up. It is a **budget that degrades as it fills**, and managing it well is most of what separates a coherent long run from a confused one. (This is the operational flip-side of Chapter 3's long-context engineering: Ch. 3 was how the *model* handles long inputs; this is how the *harness* decides what to put in them.)

> **Context engineering** (Anthropic) is choosing the smallest set of high-signal tokens to put in each call. Its central hazard is **context rot**: as the token count rises, the model's ability to accurately recall any single fact *decreases*. The older cousin is **lost in the middle** (Liu et al., 2023): models

> use information best at the *start or end* of the context and degrade sharply when it sits in the middle — even in long-context models. A bigger window does **not** buy uniform recall.

The four levers, lightest touch first:

1. **Clear old tool outputs.** The cheapest win — a 40k-token file dump from step 3 is dead weight by step 20. Drop it first.
2. **Compact / summarize.** Near the limit, summarize the conversation so far — preserving "architectural decisions, unresolved bugs, and implementation details" — and reinitialize on the summary (Claude Code's `/compact`).
3. **Retrieve just-in-time.** Hold lightweight identifiers (file paths, queries, links) and fetch content only when needed, instead of pre-loading everything.
4. **Offload to memory.** Write durable state to a notes file *outside* the window (see Component 7), so the window stays a working set, not an archive.

The failure mode to avoid is dumping a whole repo or a 200k-token file into the prompt up front: the window refills the instant it's summarized — **compaction thrash** — and the agent forgets the goal in the middle of a wall of retrieved text. Anthropic's "**Goldilocks**" **rule** for the system prompt: neither brittle hardcoded logic nor vague hand-waving, but "specific enough to guide behavior, flexible enough to give strong heuristics." Start minimal; add a rule only when you've seen the failure it fixes.

Component 4 — sub-agents & orchestration

One agent, one context window. When a task is too broad for one window, you fan out — but this is **the most expensive lever in the harness**, and it helps only for the right *shape* of work.

> In the **orchestrator–workers** pattern (Anthropic), a lead agent decomposes the task and spawns sub-agents, each in its *own isolated context window*, each returning only a "condensed, distilled summary" (~1–2k tokens) rather than its full transcript. The point is **context isolation**: five parallel explorations don't pollute the lead's window; it sees five clean summaries.

The numbers from Anthropic's multi-agent research system are the whole argument in three lines:

- A lead + sub-agents **beat a single agent by 90.2%** on their research eval.
- It burned **~15× the tokens** of a plain chat.
- **Token usage alone explained ~80% of the performance variance** — much of the win is just "more tokens spent searching in parallel."

So the decision rule is sharp. **Fan out** when the work is broad, parallelizable, and each strand is *independent*: researching many sources, searching a large codebase from several angles, reviewing a diff along several dimensions at once. **Do not fan out** tightly interdependent work — most coding, where edit B depends on what edit A just did. Anthropic is explicit: multi-agent is "less effective for tightly interdependent tasks such as coding." Fanning those out means agents step on each other and you pay $15\times$ for a *worse* result.

> Reach for sub-agents only when *outcome value dominates token cost* and the task decomposes into independent, breadth-heavy strands. Keep each worker's return small (a summary, not a transcript). Autonomy scales cost super-linearly — spend it where isolation actually buys you something.

Component 5 — permissions & sandboxing

The moment an agent can run a shell command, it can also delete your work or leak your secrets. This component is what lets you widen the leash later *without dread* — which is exactly why it comes **early** in the build order, not last.

> Two layers. **Permissions** decide *which actions need a human's sign-off*: side-effecting or irreversible actions (writes, shell, network, deploys) gated behind approval modes and allowlists, with least privilege as the default. **Sandboxing** decides *where code runs*: inside an isolated container/VM with scoped filesystem and network access, so arbitrary code can't harm the host. Permissions bound *intent*; the sandbox bounds *blast radius*.

Claude Code's four permission modes (cycled with Shift+Tab) are a clean template for **graduated autonomy** — a leash you can lengthen. Trusted commands (`npm test`, `git status`) go on an allowlist scoped org → project → personal, so the routine stuff stops asking while the dangerous stuff still gates. OpenHands runs every session in a per-session Docker sandbox so arbitrary code "can be run safely without risking the host." Reversible edits get a **checkpoint** before each write so the human can rewind.

The catastrophic anti-pattern: full auto-approve (`bypassPermissions`) on a machine with production credentials and no sandbox — one prompt-injection in a fetched web page and the agent runs `rm -rf` or exfiltrates a token, with nothing between intent and damage.

> Default to least privilege and make the *safe* path the *easy* one (allowlists beat approval fatigue). But note the boundary: checkpoints can rewind local files — they **cannot** un-send an email or un-drop a table. Route anything with an external, irreversible effect through stricter, explicit human

> approval, always.

Component 6 — verification

This is the component that turns Component 1's *open* loop into a *closed* one. Without it the agent is guessing whether it succeeded; with it, the environment tells it — and rejects bad actions before they land.

> **Verification** connects the agent's actions to real environment signals — tests, linters, type-checkers, compilers — and forces the agent to *observe the result* before proceeding. The strongest form is a **guardrail** that rejects a bad action *at the interface* rather than letting it commit and cascade. A close cousin is the **evaluator–optimizer** loop (Anthropic): one model call generates, another critiques against explicit criteria, and it iterates.

The canonical example is SWE-agent's edit tool running a linter *before* the change is written: "Python files will be checked for syntax errors after the edit. If an error is found, the edit will not be executed" — and the tool shows the error, the proposed change, and the original so the agent can recover. That one guardrail is worth about **3 points** of resolve rate ($18.0\% \rightarrow 15.0\%$ without it). **The action is rejected at the interface; the broken state never exists.** The retry discipline that pairs with it is **Reflexion** (Shinn et al.): on failure, the agent writes a short natural-language self-critique and retries with it in context.

The anti-pattern is applying a diff blind and moving on: the build breaks, the agent never learns, and it carries a false "done" into the next step where the error compounds. **"It looks right" is not verification; running it is.**

> Prefer guardrails that make a bad action *impossible to commit* over post-hoc detection. Every action should produce an observable, machine-checkable signal, and the loop should force the agent to read that signal before its next move. A harness with no verification is a very expensive way to generate confident wrong answers.

Component 7 — memory & persistence

Context (Component 3) is *working* memory, and compaction wipes it. Real long-horizon work needs state that outlives any single window — and runs you can reproduce, resume, and fork.

> Three kinds. **Scratchpad / notes:** files the agent writes and reads back, "persisted to memory outside of the context window." **Project memory:** instructions and conventions loaded into *every* session (a `CLAUDE.md` / rules file). **Session persistence:** an append-only log of the run so it can be

> resumed, forked, and reproduced. The reference architecture for long-term recall is Generative Agents' **memory stream** — store timestamped observations, retrieve by $\text{recency} \times \text{importance} \times \text{relevance}$, and periodically reflect into higher-level notes.

Durable rules live in `CLAUDE.md` (loaded into every system prompt), **not** in chat that compaction will drop — Claude Code warns that early instructions "can get lost," so a rule stated once in turn 2 is gone by turn 40. OpenHands makes the design principle exact: an **append-only event log is the state**, and "what the agent perceives is a fold over that log," so every run is reproducible.

> Anything that must survive a compaction or a restart lives on disk, not in context. Make the log the source of truth (state = a fold over events) and you get reproducibility, resume, and fork for free — the *same discipline that makes a backtest trustworthy* (Chapter 1) makes an agent run trustworthy.

Component 8 — observability & evaluation

You cannot improve what you cannot see, and you cannot trust a number that measured the wrong thing. This component makes every run legible and grades the *whole harness* honestly.

> **Observability:** every action and observation is logged, with accumulated tokens and cost, so a run can be audited step by step (OpenHands' typed event stream is exactly this — the audit log *is* the state). **Agentic evaluation:** grading the harness on tasks that require the loop, tools and environment end-to-end — *not* the base model on multiple-choice quizzes, which miss everything the harness does.

Because token usage explained ~80% of performance variance in Anthropic's multi-agent evals, **cost is a capability signal, not just a budget line**. Log tokens per run alongside success rate, or you'll ship a harness that "improved" only by quietly spending $15\times$ more. The benchmarks worth knowing: **SWE-bench Verified** and **Terminal-Bench**, with `resolve@1` reported honestly; plus per-component eval — Anthropic even lets agents "analyze your results and improve your tools for you." The anti-pattern is grading the base model on a knowledge quiz and inferring agent capability: SWE-agent showed the interface alone moves resolve rate 3–8 points at fixed model, so a static eval attributes harness bugs to the model and misses every real regression.

The meta-harness: the scaffold as a searchable artifact

Everything above is hand-authored. A research lineage asks the next question: if the scaffold matters *this much*, why design it by hand at all?

> The insight behind **ADAS** (Hu, Lu & Clune, 2024): if agents are defined in *code* — prompts, tool wiring, control flow — then the harness itself is a **searchable design space**, not a fixed thing you author once. Their *Meta Agent Search* has a meta-agent program new agents against a growing archive of prior designs; the invented scaffolds beat hand-designed ones and transfer across domains and models.

The lineage climbs from there:

- **ADAS / Meta Agent Search** (arXiv:2408.08435) — search over agents-as-code; discovered scaffolds beat hand-built ones.
- **Gödel Agent** (arXiv:2410.04444) — an agent that reads and rewrites its own code at runtime toward a high-level objective. The purest "meta harness," and a research prototype, not production.
- **Darwin-Gödel Machine** (arXiv:2505.22954) — self-modifies and *empirically validates* each change on a benchmark; self-improved from **20.0% → 50.0%** on SWE-bench. Verification (Component 6) becomes the *engine* of self-improvement.

This matters for two reasons. First, it is the strongest evidence for this chapter's thesis — DGM's $20\% \rightarrow 50\%$ swing came entirely from changing the **scaffold**, model fixed. Second, it clarifies the goal: a great harness is not a pile of features but a design space you can measure and move through. You may build it by hand for years, but knowing it's searchable changes *how* you build: instrument everything (Component 8), and every component becomes a knob you can tune with evidence instead of taste.

Build order: a runnable spine before a wide leash

Run-time order ($1 \rightarrow 8$) is **not** build order. Build the parts that let you *trust and observe* an agent before the parts that make it *powerful*:

1. **The loop + 1–2 tools** — so anything runs at all.
2. **A read tool + the environment wiring** — so turns are anchored to ground truth.
3. **Verification (Component 6) + observability (Component 8)** — so you can see what it did and whether it worked.
4. **The permission cage + sandbox (Component 5)** — so you can widen autonomy without fear.
5. **Memory & persistence (Component 7)** — so runs are durable, resumable, reproducible.
6. **Context management (Component 3)** — as runs get long enough to need it.
7. **Sub-agents (Component 4)** — last, only once a single agent is trustworthy and the task shape justifies the cost.

> **Observability and safety before autonomy.** Verification, tracing and the permission cage come *before* you widen the leash, because they are what let you extend it without fear. A powerful agent you can't observe or contain is a liability; a modest one you can trust and measure is a foundation. It is the same lesson as Chapter 1's finance harness — **the discipline is the moat, not the raw capability** — and the same closed-loop principle as Chapter 4's feedback view of learning: a system only converges when it can measure its own error against the world.

Chapter 6: Injecting an Identity — Textual Inversion, LoRA, and DreamBooth

The previous chapters were about *language* models and the machines around them. This chapter changes medium: it is about **diffusion models for images and video**, and one specific, very practical problem — **how do you make a generative model reproduce the *same subject* every time?** Prompt a fresh model with "a photo of a woman" ten times and you get ten different women. For any application that needs a *consistent character* — a recurring brand mascot, a synthetic presenter, a persistent AI persona — that randomness is fatal. The fix is a family of **fine-tuning** techniques that *inject a new identity* into a pretrained model: **textual inversion**, **LoRA**, and **DreamBooth**. They sit at three different points on the same axis — *how much of the model you are allowed to change* — and understanding that axis explains their fidelity, their cost, and their failure modes. The chapter closes with the **image→video pipeline** (where identity is actually solved) and the **open-weight model landscape and economics** as of late 2026.

The framing here is media generation, but the underlying lesson is the same one that runs through [[the-agent-harness]] and the RL chapter: **you rarely retrain the whole model; you attach a small, reversible delta and protect the base.** That is exactly what LoRA is — the same low-rank-adapter idea used to cheaply specialise an LLM, applied to a diffusion U-Net.

The problem: a generative model has no notion of "her"

A text-to-image diffusion model maps a text prompt to an image by iteratively denoising random noise, guided by a **text encoder** (which turns words into conditioning vectors) and a **denoiser** (a U-Net or diffusion transformer, which does the actual drawing). The model knows *concepts* — "woman", "beach", "portrait" — as regions of its learned weight space. It does **not** know any *specific individual* unless that person was so heavily represented in training that a name retrieves them.

So to get a consistent invented character we must **teach the model a new concept** and bind it to a handle we can type in a prompt. There are three ways to do this, differing in *what* they update:

Method	What is trained	What stays frozen	Output	Fidelity
Textual Inversion	one (or a few) **embedding vectors** — a new *word*	all model weights	a tiny `.pt` / `.safetensors` embedding (KB)	low–medium
LoRA	a small **low-rank slice of the weights**	the base weights + (usually) the text encoder	a small adapter (MBs)	high
DreamBooth	the **whole model's weights**	nothing (optionally the text encoder)	a full model (GBs)	highest

The three form a ladder of *capacity*: textual inversion can only *re-describe* the subject using ability the model already has; LoRA and DreamBooth can *add new ability* to render detail the model could not produce before.

Textual inversion: train the word, freeze the model

Textual inversion asks a deliberately narrow question: *holding the entire model fixed, what single point in the text encoder's embedding space best summons this subject?*

Concretely, you introduce a new token — call it `sksgirl` — and give it a fresh embedding vector $v \in \mathbb{R}^d$ (where d is the encoder's embedding dimension, e.g. 768 or 2048). This vector is often **initialised from a seed word** ("woman") or at random, then **optimised** so that generations conditioned on it match your training images. The optimisation only ever touches v:

$$v^{*} = \arg\min_{v} \mathbb{E}_{x,\epsilon,t}\left[\left\| \epsilon - \epsilon_\theta\big(x_t,\ t,\ c(v)\big) \right\|^2 \right], \qquad \theta \text{ frozen.}$$

Here ϵ_θ is the frozen denoiser, x_t a noised training image at step t, ϵ the noise it must predict, and $c(v)$ the conditioning that contains our new token's embedding. **Only v moves.**

This is powerful precisely because it is *so* small (you are learning $\sim d$ numbers, a few KB), which makes embeddings trivially shareable and composable. But it is also its ceiling: a single vector can only *point at combinations of things the frozen model already knows how to draw*. If the subject has features outside the model's existing range, no embedding can conjure them — the model has no weights free to learn them. Hence: good for *style/vibe*, weak for a *precise unique face*.

> Textual inversion learns a **word**, not an ability. It re-describes the subject in the model's existing vocabulary; it cannot expand that vocabulary.

LoRA: train a small, reversible slice of the weights

LoRA (Low-Rank Adaptation) moves one rung up the ladder: it *does* change the weights, but only along a cheap, low-dimensional, **removable** correction.

The insight is that the *update* a fine-tune applies to a big weight matrix $W \in \mathbb{R}^{m \times n}$ is empirically **low-rank** — it can be well approximated by the product of two thin matrices. So instead of learning a full ΔW (which is $m \times n$ numbers), LoRA freezes W and learns

$$W' = W + \Delta W, \qquad \Delta W = BA, \quad B \in \mathbb{R}^{m \times r},\ A \in \mathbb{R}^{r \times n},\ r \ll \mathrm{m}$$

With rank r small (say 4–128), the trainable parameter count drops from $m\,n$ to $r\,(m+n)$ — often a 100–$1000\times$ reduction. Those A, B matrices *are* the LoRA file (a few MBs). At generation time you **add** BA back into the frozen W; unload the file and the base model is byte-for-byte its original self.

Two consequences matter for a production persona pipeline:

1. **The base is never corrupted.** A bad LoRA is a bad *add-on you delete*, not a ruined model. Contrast DreamBooth, which overwrites the base in place.
2. **Personas are tiny and swappable.** Fifteen characters = fifteen small adapters stacked on *one* shared base, not fifteen multi-GB models. You can even stack a **face LoRA** and an **action LoRA** at once.

Crucially, in a standard LoRA the **text encoder is frozen** — so the trigger word `sksgirl` is *not* given a new trained vector. It is simply **tokenised into existing sub-word pieces** (`sks` + `girl`, etc.) whose embeddings already exist, and the LoRA teaches the *denoiser weights* to map that fixed text signal onto the subject. (A variant, *text-encoder LoRA*, also adapts the encoder, but U-Net-only is the default.) This is the key distinction people blur:

> **Textual inversion updates the word and freezes the weights; LoRA freezes the word and updates the weights.** The random-initialised-new-vector intuition is correct — *for textual inversion*. In a LoRA the word is a fixed handle and all the learning lives in the low-rank weight delta. That extra capacity is why LoRA captures a specific face far better than an embedding can.

DreamBooth and the catastrophic-forgetting problem

DreamBooth is the top rung: fine-tune **all** the model's weights on the subject. With the most capacity it reaches the highest fidelity — but it exposes a failure mode the other two mostly dodge, and its real contribution is the *fix* for that failure.

The failure — catastrophic forgetting / language drift. Weights in a diffusion model are *shared* across concepts: the same parameters that draw "woman" also serve "person", "face", "portrait", even "man". If you hammer the full model with only ~ 20 images of one subject, gradient descent happily overwrites those shared weights to fit her — and drags the *neighbours* along. Symptoms: every "woman" you later generate drifts toward her face, and the trigger word's original meaning decays. You have told the model *the entire concept of woman is now this one person*.

The fix — prior preservation. Before training, you use the *current, un-tuned* model to generate a batch (typically 100–200) of generic **class images** — "a photo of a woman" — that capture what the model *already* believes the class looks like. These are the **regularisation set**. Then you train on two interleaved streams and add their losses:

$$\mathcal{L} = \underbrace{\left\|\epsilon - \epsilon_\theta(x_t^{\text{inst}},\, t,\, c_{\text{"sksgirl"}})\right\|^2}_{\text{instance loss — learn HER}} + \lambda \underbrace{\left\|\epsilon - \epsilon_\theta(x_t^{\text{class}},\, t,\, c_{\text{"woman"}})\right\|^2}_{\text{prior-preservation loss — keep the class inta}}$$

The second term is an **anchor**: every step that tries to pull "woman" toward her, the regularisation stream pushes back with "no — `woman` must still match these 200 diverse examples I generated a moment ago." The weight λ sets how hard the anchor pulls. The net effect is that the new identity attaches to the **fresh, walled-off trigger word** while the general class is held in place.

```
Without prior preservation:  train on her only        → "woman"
collapses to her
With prior preservation:     train on her + class set → sksgirl = her,
woman = all women
```

The two-stream training loop — input and output

Both streams run in the *same* optimisation loop; they are just two kinds of labelled examples mixed into each batch:

```
INPUTS:
  instance set:  ~15–30 photos of HER      + caption "a photo of
sksgirl"
  class set:     ~100–200 model-generated  + caption "a photo of a
woman"
                 generic "woman" images

PER STEP (for each image x, whether instance or class):
  t   ~ Uniform(timesteps);   ε ~ N(0, I)              # random noise
level + noise
  x_t = √(ᾱ_t)·x + √(1−ᾱ_t)·ε                          # add noise to
the latent
  ε̂  = ε_θ(x_t, t, c)                                  # model
PREDICTS the noise (given caption c)
  loss_image = ‖ε − ε̂‖²                                # error on the
NOISE, not on x

  loss_her   = loss_image for an instance latent  (c = "sksgirl")
  loss_class = loss_image for a class latent       (c = "woman")
  total      = loss_her + λ · loss_class
  backprop(total) → update weights (full model for DreamBooth, low-
rank BA for DreamBooth-LoRA)

OUTPUT:
  DreamBooth        → a whole new model file (GBs)
  DreamBooth-LoRA   → a small adapter (MBs)
```

How the diffusion loss is actually computed

The pseudocode above is loose in one important way, and it is worth being exact: **the loss is never "how far the output image drifts from her photo."** A diffusion model is not trained to output a finished image in one shot — it is trained to **predict the noise** (equivalently, the *velocity*) that was added to a partially-noised sample. The per-example objective is the **denoising score-matching loss**:

$$\mathcal{L}_{\text{image}} \;=\; \mathbb{E}_{t\sim\mathcal{U},\;\epsilon\sim\mathcal{N}(0,I)}\Big[\;\big\|\,\epsilon \;-\; \epsilon_\theta\big(\underbrace{\sqrt{\bar\alpha_t}\,x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon}_{x_t},\; t,\; c\big)\,\big\|^2\;\Big].$$

Read it step by step:

1. **Take a clean training latent** x_0 (the VAE encoding of the photo).
2. **Pick a random timestep** t and **sample Gaussian noise** ϵ.
3. **Corrupt it** to $x_t = \sqrt{\bar\alpha_t}\,x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon$, where $\bar\alpha_t$ is the noise schedule (at small t barely noised, at large t almost pure noise).
4. **Ask the model to predict the noise**: $\hat\epsilon = \epsilon_\theta(x_t, t, c)$, conditioned on the caption c (`"sksgirl"` or `"woman"`).
5. **The loss is the error on the noise**, $\|\epsilon - \hat\epsilon\|^2$ — a per-element MSE in *latent* space, **not** a comparison of rendered images.

So when we said "*loss_her = how far output drifts from her photo*" that was shorthand: the gradient signal is "*given this caption, at this noise level, did you correctly identify the noise on **her** latent?*" Learning to denoise her latents *across all timesteps* is what binds `sksgirl` to her — and doing the same for generic `woman` latents is what the prior-preservation term protects.

Two notes on modern variants:

- **Velocity / flow-matching parameterisation.** Newer models (including rectified-flow systems like Flux and several video models) don't predict ϵ directly; they predict a **velocity** v — the target is $v = \alpha_t \epsilon - \sigma_t x_0$ (or, in flow matching, the straight-line drift $x_1 - x_0$ between noise and data). The *shape* is identical — an MSE between a predicted and a target vector, $\|v - v_\theta(x_t, t, c)\|^2$ — only the regression target changes. Every conclusion in this chapter is unchanged; substitute v for ϵ.
- **Why it's noise, not pixels.** Training on the final image would require running the full multi-step sampler *inside* every gradient step (slow, unstable). The score-matching trick reduces the whole generative problem to **one-step supervised regression at a random noise level** — cheap, and what makes training these models tractable at all.

Two details that make or break identity fidelity in practice:

- **Caption only what varies.** Tag the *changeable* content of each instance image (pose, outfit, background, lighting) but **never describe her face**. The trigger word is constant across all instance images, so the model binds it to whatever is *also* constant — her face/body. If you caption "blue eyes, small nose", the model treats those as optional variables and identity goes fuzzy.
- **Use varied angles.** Front, three-quarter, profile; close-up and full-body; multiple expressions and lighting. Uniform selfies teach a flat, frontal identity that warps off-axis; variety teaches the *3-D* identity.

Why LoRA is the practical winner — and DreamBooth-LoRA the real answer

LoRA does **not** magically escape forgetting — a carelessly trained LoRA can still bleed the subject into the class, which is why good LoRA training *also* uses regularisation images. But the damage is bounded: the base is frozen and reloadable, and the low-rank footprint has little capacity to wreck neighbours. Weigh the three:

	Full DreamBooth	LoRA (w/ prior preservation)
Base model	**overwritten in place**	untouched; adapter loaded on top
Output size	whole model per subject (GBs)	small file per subject (MBs)
Fidelity	highest	very high — gap is small
Blast radius if training fails	a ruined model	delete a file
Swapping N personas	load N big models	stack N tiny adapters on one base

The field settled on a hybrid: run **DreamBooth's *method*** (trigger word + prior preservation) but restrict the update to a **LoRA-sized slice** — i.e. **DreamBooth-LoRA**. You inherit DreamBooth's training quality *and* LoRA's small, reversible, swappable output. DreamBooth is the *technique* (how to inject a subject without breaking the model); LoRA is the *output format*. Best practice combines them.

Where identity is actually solved: the image→video pipeline

A subtle but decisive point for video: **you do not need an identity fine-tune on the video model at all**, because identity is solved one stage earlier. The pipeline splits into two jobs:

1. **Make a consistent face (image stage).** A **LoRA on the image model** (Flux / SDXL / Pony) produces stills of the same person in any pose. This is where the identity work lives.
2. **Animate it (video stage, image-to-video).** Feed a finished still as the **first frame** to a video model in **image-to-video (I2V)** mode. The model animates *that exact image* — so the identity is carried **by the input frame**, not learned by the video model. The video model never has to "remember" her.

```
Flux/Pony + face-LoRA   →  consistent STILL  ──(as I2V start frame)──►
Wan/LTX  →  animated CLIP of the same face
```

A LoRA on the *video* model is therefore only for what the input frame cannot carry: **specific motions/actions** (an action LoRA) or pure text-to-video with no reference. To extend a clip, chain it: take the **last frame** of clip N as the **first frame** of clip $N{+}1$ (first/last-frame conditioning), then concatenate with `ffmpeg`. No current open model generates minutes of coherent video in one shot; long output is always *assembled* from short, identity-anchored clips.

The open-weight landscape and the economics of "adding a capability"

Two things about the late-2026 open video-model landscape matter for anyone planning to self-host:

- **Open weights = you control everything.** "Open-weight" means you download the actual weight files and run them yourself (e.g. in ComfyUI) on your own or a rented GPU. There is no API, no content filter, and no account to ban — the filtering that hosted APIs apply lives *outside* the weights. This is the entire reason specialised communities build on open models: capability and control.
- **Newer is not always usable. Wan 2.5** (Alibaba, Sep 2025) is *API-only* — its weights are not released — so for self-hosting, **Wan 2.2** is the newest *open* Wan. **LTX-2.5** (LTX, Aug 2026, $\sim$22B params) *is* open-weight, very fast, lighter on VRAM, and supports multi-shot identity preservation — but its community fine-tune ecosystem is younger than Wan's. **Hunyuan Video** (Tencent) sits between them, strong on motion/physics.

Model	Open weights?	Photorealism	Best at	VRAM
Wan 2.2	yes	★★★★★	facial detail, skin/hair; largest fine-tune ecosystem	high (24 GB+ ideal)
Hunyuan Video	yes	★★★★½	natural motion & physics	high
LTX-2.5	yes	★★★★	speed, multi-shot identity, lighter GPUs	**lowest**
Wan 2.5	**no (API only)**	★★★★★	1080p, synced audio	n/a (hosted)

(An easy on-ramp for small GPUs is a quantised/offloaded runner such as `Wan2GP` *— the same Wan weights repacked to fit $\sim$8–12 GB cards: same output, slower, not a weaker model.)*

The economics — and why you almost never fine-tune a base yourself. There is a sharp cost cliff between *teaching one subject* and *adding a whole capability*:

- **A character/action LoRA** (one subject or one motion) trains in hours on a rented GPU. At late-2026 rates — RTX 4090 $\approx$ $0.30–0.70/hr, A100 $\approx$ $1–2/hr, H100 $\approx$ $2–3/hr — an **image face-LoRA** is $< \$2$, a **video LoRA** roughly **$5–$40**.
- **Re-teaching a broad capability to a base** (hundreds–thousands of curated, captioned clips) is an *order-of-magnitude* jump: a broad concept LoRA might run **$100–$500** in compute; a **full fine-tune of a 22B video base** runs **multi-GPU for days → roughly $1,000–several thousand**, plus serious

data engineering. These figures are *order-of-magnitude estimates* — `GPU-hours × rate`, with wide error bars — not measurements; the honest way to pin your own number is a small **pilot** (train on $\sim$50–100 clips for $\sim$ $20–50, inspect, then extrapolate).

> The dataset — sourcing and captioning hundreds to thousands of examples — is the real bottleneck, not the GPU bill. And you rarely pay the big number at all: one team fine-tunes the base capability once and **publishes the checkpoint for free**; everyone else downloads it and layers **cheap per-subject LoRAs** on top. It is the same division of labour as pretraining vs. fine-tuning an LLM — inherit the expensive base, own only the small delta. Building a *proprietary* base buys little over a mature free checkpoint unless the base itself is your moat.

The through-line back to the rest of this book: whether the medium is language or pixels, the winning move is almost never "retrain the whole model." It is **attach a small, reversible, well-regularised delta to a strong frozen base, and protect that base** — LoRA for a persona here, an adapter for a domain LLM elsewhere, the disciplined scaffold around the weights everywhere.

Chapter 7: Manufacturing the Right Signal — MoE Load Balancing and Lean-Verified Proofs

Two problems in this chapter look unrelated — *how do you stop a sparse Mixture-of-Experts model from routing every token to the same three experts?* and *how does an AI solve olympiad mathematics?* — but they are the same lesson twice. In both, the naive objective you actually care about is **impossible to optimise directly**: it is either non-differentiable (a token count) or unverifiable (a chain of mathematical reasoning). The fix in both cases is to **manufacture a cheap, well-behaved proxy signal** — a differentiable balance term in one case, an exact mechanical reward in the other — and optimise *that*. The proxy is the whole game. This chapter derives the MoE **load-balancing loss** in full, then follows the second idea into **Lean-verified theorem proving** and the **AlphaZero → AlphaProof → AlphaEvolve** lineage of systems built on a perfect automatic referee.

The connection to the rest of the book: the load-balancing loss is the missing mechanism behind every sparse frontier LLM, and Lean-verified RL is the pure form of the *reinforcement learning with verifiable rewards* first met in [[code-world-models]] — here the verifier is not a unit test but a proof kernel.

Part A — The Mixture-of-Experts load-balancing loss

The problem: routing collapse

A Mixture-of-Experts (MoE) layer replaces a dense feed-forward block with N parallel expert FFNs and a small **router** (gating network). For each token the router picks the top-k of the N experts (Switch Transformer uses $k = 1$), and *only those experts run*. That is the entire economic case for MoE: you multiply the parameter count by N while keeping the per-token FLOPs roughly constant.

The failure mode is a positive-feedback loop called **routing collapse**. Early in training a few experts are marginally better, so the router sends them more tokens, so they receive more gradient and improve faster, so the router sends them *even more*. Left alone, a 64-expert layer converges to maybe 4 live experts and 60 dead ones — you paid for capacity you never use, and the fixed per-expert **capacity buffers** overflow, silently dropping the overflow tokens. The load-balancing loss is a regulariser that breaks this loop.

Setup and notation

For a batch of T tokens and N experts, the router emits a logit $h_i(x)$ for each expert i on token x, then a softmax:

$$p_i(x) \;=\; \frac{e^{h_i(x)}}{\sum_{j=1}^{N} e^{h_j(x)}}.$$

$p_i(x)$ is the router's **soft** probability of dispatching token x to expert i. The actual dispatch is the **hard** top-k selection. Now define two per-expert batch statistics:

$$f_i \;=\; \frac{1}{T}\sum_{x} 1\{\text{token } x \text{ routed to expert } i\}, \qquad P_i \;=\; \frac{1}{T}\sum_{x} p_i(x).$$

f_i is the **hard load** — the fraction of tokens actually dispatched to expert i, a histogram built from an $\arg\max$. P_i is the **soft load** — the mean router probability mass on expert i. The entire construction hinges on the difference between these two: f_i is a *count*, P_i is a *smooth average*.

The loss

The Switch Transformer / GShard auxiliary loss is

$$\boxed{\mathcal{L}_{\text{aux}} \;=\; \alpha\, N \sum_{i=1}^{N} f_i\, P_i}$$

- The factor N normalises the minimum value to 1 regardless of expert count, which makes the coefficient α transferable across model sizes.
- α is small, typically $\alpha \approx 10^{-2}$: large enough to enforce balance, small enough not to dominate the language-modelling objective.

Where it is computed, and what it is added to

This is the question that trips people up. The loss is computed **inside every MoE layer** — each layer has its own router, so its own f_i, P_i — then summed across all MoE layers and added to the main next-token cross-entropy as a single scalar:

$$\mathcal{L}_{\text{total}} \;=\; \underbrace{\mathcal{L}_{\text{CE}}}_{\text{language modelling}} \;+\; \alpha \sum_{\ell \,\in\, \text{MoE layers}} \mathcal{L}_{\text{aux}}^{(\ell)}.$$

So it is **not** attached to a "final layer." It is a global auxiliary term on the overall training loss, and *every router in the network* receives gradient from it. Backprop of $\mathcal{L}_{\text{CE}}$ trains "route tokens to experts that predict well"; backprop of $\mathcal{L}_{\text{aux}}$ trains "and spread the load evenly." They are summed and optimised jointly, exactly like a weight-decay or KL term.

Why the specific form $\sum_i f_i P_i$ — the differentiability trick

You would *like* to penalise imbalance directly — say, minimise the variance of f_i. But f_i comes from an $\arg\max$: it is a **non-differentiable count**, so no gradient flows through it. You cannot train the router by pushing on f_i.

P_i, by contrast, is a smooth softmax average — **fully differentiable**. The product $f_i P_i$ resolves the tension by assigning each factor a distinct role:

- f_i is a **non-differentiable weight** — "how overloaded is expert i right now," treated as a constant by the optimiser.
- P_i is the **differentiable carrier** — the knob the router can actually turn.

Differentiate with respect to the router's soft mass on expert i:

$$\frac{\partial \mathcal{L}_{\text{aux}}}{\partial P_i} \;=\; \alpha\, N\, f_i.$$

Read literally: the downward pressure on expert i's routing probability is **proportional to how overloaded it already is**. Overloaded experts (f_i large) get their probability suppressed hard; starved experts ($f_i \approx 0$) get almost no penalty, freeing the router to send them more. That is precisely the corrective signal we wanted from the variance objective — but delivered through the differentiable P_i.

Why the minimum is the balanced state

Both f and P are distributions on the simplex ($\sum_i f_i = 1$, $\sum_i P_i \approx 1$). The loss is proportional to the **dot product** $\langle f, P \rangle$. Subject to those constraints, the dot product of two distributions is *largest* when both are spiky and aligned on the same experts — which is exactly collapse — and *smallest* when both are flat. The minimum sits at the uniform point $f_i = P_i = 1/N$:

$$\alpha\, N \sum_{i=1}^{N} \frac{1}{N} \cdot \frac{1}{N} \;=\; \alpha\, N \cdot N \cdot \frac{1}{N^2} \;=\; \alpha.$$

(The α-free part equals 1 — the normalisation the factor N was chosen to produce.)

Worked example

Take $N = 4$ experts and $T = 100$ tokens, and set $\alpha = 1$ to read the raw term. Compare a **collapsed** batch against a **balanced** one.

Collapsed: the router dumps most tokens on expert 1.

$$f = (0.70,\, 0.20,\, 0.10,\, 0.00), \qquad P = (0.60,\, 0.20,\, 0.15,\, 0.05).$$

$$\mathcal{L}_{\text{aux}} = 4\big(0.70{\cdot}0.60 + 0.20{\cdot}0.20 + 0.10{\cdot}0.15 + 0.00{\cdot}0.05\big) = 4(0.475) =$$

Balanced: $f = P = (0.25, 0.25, 0.25, 0.25)$.

$$\mathcal{L}_{\text{aux}} = 4\big(4 \times 0.25 \cdot 0.25\big) = 4(0.25) = 1.00.$$

The collapsed configuration scores 1.90 against the balanced floor of 1.00 — a 90% penalty the optimiser can shrink only by flattening the distributions. And because the gradient on P_1 is $\alpha N f_1 = 4(0.70) = 2.8$ versus 0 on the empty

expert 4, the very next step pushes probability away from the crowded expert toward the empty one.

> **The load-balancing loss manufactures a differentiable signal out of a non-differentiable one.** f_i (a count, no gradient) becomes a *weight* on the differentiable P_i, so minimising the dot product $\langle f, P \rangle$ pushes the router toward uniform utilisation — and the gradient on each expert is exactly proportional to how overloaded it is.

Two companions you meet in real code

Router z-loss (ST-MoE) keeps the logits from exploding, which stabilises the softmax and sharpens routing:

$$\mathcal{L}_z = \frac{1}{T} \sum_{x} \left(\log \sum_{i=1}^{N} e^{h_i(x)} \right)^2 .$$

It penalises a large log-sum-exp — i.e. large logits — and is added with its own small coefficient alongside $\mathcal{L}_{\text{aux}}$.

Auxiliary-loss-free balancing (DeepSeek-V3) is the current frontier trend. Instead of adding $\mathcal{L}_{\text{aux}}$ — which slightly degrades quality by fighting the language objective — it adds a **per-expert bias** b_i to the score used *only for top-k selection*:

$$\text{select top-}k \text{ of } \big(h_i(x) + b_i\big).$$

The bias is **not trained by gradient descent**. A simple control rule nudges it after each step: if expert i was overloaded, decrease b_i; if starved, increase it. This achieves balance without ever perturbing the main-loss gradient — the "balance tax" on model quality disappears. It is a nice illustration that a manufactured signal need not be a *loss term*; here it is a control-loop feedback on a routing bias.

Part B — Verifying mathematics with Lean

A verifier that cannot be fooled

The second half of the chapter swaps domains but keeps the theme. **Lean** is an interactive theorem prover built on dependent type theory, where a deep correspondence (Curry–Howard) holds:

$$\textbf{a proof} \equiv \textbf{a term}, \qquad \textbf{a theorem} \equiv \textbf{that term's type}.$$

Proving "$\sqrt{2}$ is irrational" means *constructing a term whose type is the formal statement*. A tiny, heavily-audited **kernel** then type-checks that term. If it checks and contains no `sorry` placeholder, the theorem is true — modulo trusting a few thousand lines of kernel.

The consequence for machine learning is enormous: you get a **binary, mechanical, zero-false-positive reward**. The proof compiles or it does not. No human grader, no "looks plausible," no fuzzy LLM-judge to reward-hack. Contrast informal competition math with a numeric answer, where the only check is *did the final number match* — the reasoning in between can be nonsense and still land right. Lean verifies the **entire chain**. This is the strongest possible form of the verifiable reward from [[code-world-models]]: there the oracle was a unit test; here it is a soundness-guaranteeing proof kernel.

The tactic REPL, and what LeanDojo does

An agent does not emit a whole proof blind. Lean exposes a **proof state** — the current goal plus hypotheses. The model proposes a **tactic** (a step such as `induction n` or `rw [lemma]`); Lean executes it and returns the *new* state, either new subgoals or an error. The proof is a path:

```
state_0  --tactic_1-->  state_1  --tactic_2-->  ...  -->  no goals left  (QED)
```

Lean by itself is a tool for a human at an editor. To let a *program* drive this loop you need an API, and that is **LeanDojo** (an open-source toolkit and benchmark, Caltech-led, 2023). It does three distinct jobs:

1. **Tracing / data extraction** — it parses Lean and its `Mathlib` library into structured `(proof state, tactic, next state)` triples, plus which library lemmas ("premises") each step used. This turns all of Mathlib into supervised training data.
2. **An interaction gym** — a Gym-style environment (`reset` to a theorem, `run_tac(state, tactic)`, observe the result) that exposes the REPL loop above in a reproducible sandbox. This is the piece a search algorithm actually drives.
3. **A benchmark and baseline** — the LeanDojo Benchmark plus **ReProver**, a *retrieval-augmented* prover that retrieves likely-relevant lemmas from Mathlib's hundreds of thousands before generating each tactic, so the model need not memorise the whole library.

> LeanDojo is **infrastructure**, not a champion solver — the plumbing that converts Lean from a human editor into a machine-drivable environment with training data and a benchmark. Research systems are built *on top of* it.

RL with verifiable rewards, and expert iteration

Because the kernel's verdict is free and exact, the reward is simply

$$R(\text{proof}) = \begin{cases} 1 & \text{Lean kernel accepts, no } \texttt{sorry} \\ 0 & \text{otherwise.} \end{cases}$$

That single fact makes reinforcement learning tractable and enables **expert iteration** (the AlphaProof recipe): (1) auto-formalise a large problem corpus into Lean statements; (2) let the current model search for proofs; (3) **keep every proof Lean verifies** as new gold training data — no human labelling; (4) fine-tune on the verified proofs, producing a stronger model that cracks harder problems, yielding more verified proofs. The loop bootstraps unlimited, guaranteed-correct training data — the scarce resource in mathematics.

What the traced data actually trains — and at which stage

It is worth being precise about *why* the tracing step exists and *what* it feeds, because it is easy to assume the Lean proofs train the base model. They do not — the training happens in three separate stages, mapping exactly onto the pipeline in [[code-world-models]].

Stage 1 — Base pretraining (not this data). The foundation model (Gemini for AlphaProof; an off-the-shelf model such as Google's ByT5 for the academic ReProver) is pretrained on the usual web-and-code corpus. This is what gives it general language, mathematics, and code fluency. The structured traced triples are *not* what builds it. (Public `Mathlib` lives on GitHub, so some raw Lean source likely leaks into any web scrape — but that is incidental exposure, not the structured `(state, tactic)` supervision.)

Stage 2 — Supervised fine-tuning into a "prover" (this is the tracing data). Every proof in Mathlib was written by a human, so tracing recovers, for each step, *the goal state at that moment* (the input) and *the tactic the human applied* (the label). Fine-tuning the base model on these `(state → tactic)` pairs is **behavior cloning**: the model learns "when a goal looks like this, experts tend to do that." This is the same slot as the **SFT** stage in [[code-world-models]] — "putting good behaviours within reach." Separately, the "which lemmas each step used" annotation gives `(goal → relevant premises)` pairs that train the **retriever** (ReProver), because Mathlib's hundreds of thousands of lemmas cannot all fit in context.

Stage 3 — RL with verifiable rewards. *Then* the expert-iteration loop above runs on top, using Lean's $0/1$ verdict to push past what imitation alone reaches.

Why the SFT stage is not optional: the tactic action space is effectively unbounded — any tactic, with arguments referencing any of $\sim 10^5$ lemmas — and the reward is sparse, firing only when an *entire* proof closes. A policy starting from random would essentially never stumble onto a complete proof, so RL would have nothing to reinforce. Behavior cloning on traced human proofs supplies a competent **prior** that makes the search land on proofs often enough for RL to get traction. It is the AlphaGo pattern — imitate human games first, then self-play — and it only works because pretraining already gave the model the mathematical fluency the clone step builds on.

> The traced Lean proofs are the **fine-tuning (SFT) / imitation data**, not the base pretraining corpus. Pretraining gives general math and code fluency; the Lean fine-tune gives the *specific skill* of driving the tactic REPL; RL then improves on that prior. You need each stage before the next is even learnable.

Part C — The "Alpha" lineage: AlphaZero → AlphaProof → AlphaEvolve

The expert-iteration loop above is not new; it is a domain swap on a template DeepMind established in games. Three systems are worth placing on one family tree, because they are constantly confused.

AlphaZero (2017) — the ancestor idea

AlphaZero mastered Go, chess, and shogi *from scratch* — no human games, only the rules — with a recipe every later "Alpha" system reuses:

- one neural net outputs a **policy** (which moves look promising) and a **value** (how good is this position);
- that net **guides a tree search** (MCTS) rather than brute force;
- training is pure **self-play**, and — critically — the **rules of the game are a perfect automatic referee** that declares the winner, supplying ground-truth reward with zero human labels.

Lineage: AlphaGo (2016, learned from human games) → AlphaGo Zero (2017, self-play, Go only) → AlphaZero (2017, one algorithm, three games) → MuZero (2019, also learns the rules). The reusable insight: **learned policy/value + search + a perfect automatic referee** bootstraps superhuman skill. Now swap the game.

AlphaProof (2024) — AlphaZero where the game is proving

AlphaProof makes exactly that swap, and the mapping is one-to-one:

AlphaZero (games)	AlphaProof (math)
game position	Lean proof state
legal moves	tactics
policy/value network	a language model (Gemini-based) proposing/scoring tactics
perfect referee = game rules	**perfect referee = the Lean kernel**
self-play	attempt a huge problem bank, keep verified proofs

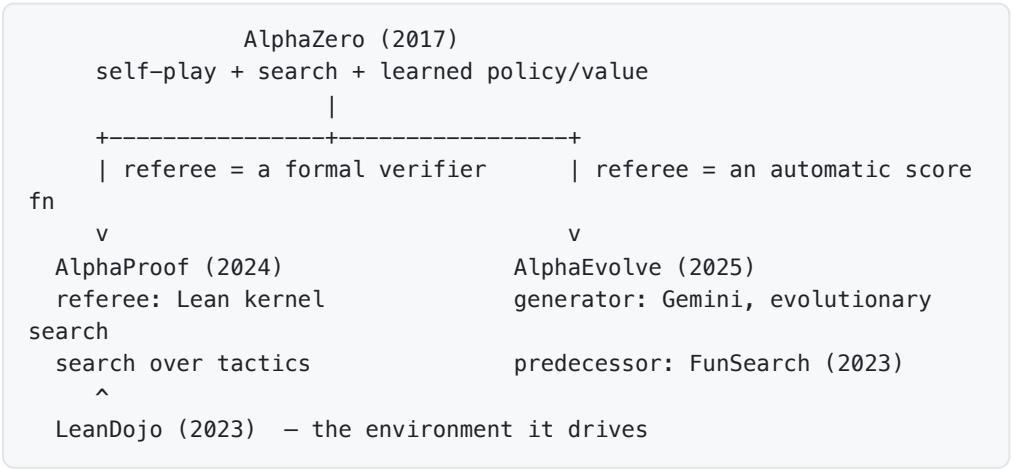
Concretely, a **formalizer** network auto-translates on the order of a million natural-language problems into Lean statements, and a **solver** searches for proofs, trained by AlphaZero-style RL against Lean's yes/no verdict. At test time it does something striking: it **generates variations** of the target problem and tries to prove those too, a self-reinforcing loop that sharpens it on the specific question in front of it. Result: at **IMO 2024** it solved 3 of 6 problems (paired with **AlphaGeometry 2** on the geometry problem) for 28/42 — **silver-medal** level, one point below gold. Every answer was a Lean-verified proof.

> **AlphaProof is the AlphaZero recipe with the Lean kernel playing the role the game rules played in Go.** Same self-play + search + learned policy/value — different referee. LeanDojo-style environments are what make the "board" drivable.

AlphaEvolve (2025) — a cousin, not a sibling

AlphaEvolve is where the confusion peaks. It shares the family DNA — *an LLM in a loop with an automatic evaluator* — but it is a **different method for a different job**. It is an **evolutionary coding agent**: Gemini proposes edits to a program, an automatic **evaluator scores** each candidate, the best survive and are mutated further — genetic programming with an LLM as the mutation operator. It descends from DeepMind's earlier **FunSearch** (2023). It does not prove theorems; it *evolves algorithms and code* for any problem you can score automatically. Its wins include multiplying two 4×4 matrices with **48 multiplications** (beating Strassen's 49 from 1969), improved bounds on open problems such as the kissing number in 11 dimensions, and real Google-infrastructure gains (data-centre scheduling, TPU circuit tweaks, a $\sim 23\%$ speedup of a Gemini training kernel).

The family tree, then, is a single idea — *a generator plus an automatic ground-truth evaluator, looped* — branching by **what plays the referee**:



The relationships in one breath: **AlphaProof ↔ AlphaZero** — same algorithm family, direct descendant. **AlphaProof ↔ AlphaEvolve** — cousins, same "LLM + auto-verifier loop" philosophy and lab, but different techniques (RL proof-search vs. evolutionary code-search) and domains. **LeanDojo** sits underneath the math branch as infrastructure, not a competitor.

> Every system in this chapter turns an intractable objective into a workable one by **manufacturing the right signal**. MoE conjures a *differentiable* balance signal ($f_i P_i$) from a non-differentiable count. Lean conjures an *exact, verifiable* reward from otherwise-ungradeable reasoning. The Alpha family shows the payoff at scale: once you have a cheap, perfect referee, a generator can bootstrap its own superhuman training data. The cheap-but-precise signal is the whole game — the same moral as the verifiable-reward RL in [[code-world-models]] and the disciplined scaffolding of [[the-agent-harness]].

Chapter 8: Not Forgetting — Catastrophic Forgetting and the SFT → RL → SFT Loop

Reinforcement learning is wonderful at *amplifying* one behaviour and terrible at *preserving* everything else. Push a pretrained model hard on a single rewarded skill — call it A — and it does get much better at A; but the same parameters that encode A also encode the model's other capabilities B, C, D, E, and those quietly degrade. This chapter is about the engineering that stops a specialised model from becoming a *narrow* one: the **consolidation dataset**, the three preservation knobs (**KL**, **replay loss**, **post-RL SFT**), and the full $\mathrm{SFT} \to \mathrm{RL} \to \mathrm{SFT}$ loop that frontier labs actually run. The central tension is between **exploration** (letting RL move far enough to discover something genuinely new) and **preservation** (not forgetting what you already knew) — and the whole recipe is an attempt to sit between the two. It is the same catastrophic-forgetting problem met in the vision setting of [[dreambooth]], now in language-model post-training, and it extends the RL / entropy-collapse machinery of [[code-world-models]].

The forgetting phenomenon: what RL on one skill does to the others

Start from a model that has been supervised-fine-tuned to be competent across a spread of skills:

$$\mathrm{SFT}_1: \quad A, B, C, D, E \text{ all usable.}$$

Now run RL with a reward that only measures A (say, passing a code test, or a verifier accepting a proof — the verifiable-reward setting of [[code-world-models]]). The gradient only ever points "towards more A," so the update drifts the shared parameters into a region that is excellent for A and incidentally worse for the rest:

$$\mathrm{RL\ on}\ A: \quad A \uparrow\uparrow, \qquad B, C, D, E \downarrow .$$

This is **catastrophic forgetting** — the continual-learning failure where learning a new task overwrites the representations an old task relied on. It is not a bug in the reward; it is a direct consequence of *one objective* steering *shared weights*. The naive fix — "just SFT on B, C, D, E afterwards to bring them back" — has a symmetric failure: train only on the old skills and you pull the parameters back out of the A-region, so now

$$\mathrm{SFT\ on}\ B, C, D, E\ \mathrm{only}: \quad A \downarrow, \quad B, C, D, E \uparrow .$$

You have merely swapped which capability you sacrificed. The problem is never "how do I train A" or "how do I restore B, C, D, E" in isolation; it is how to hold *both* in the same set of weights at once.

The consolidation dataset: a mixture, never the winners alone

The repair stage after RL is a supervised phase usually called **consolidation SFT** or **SFT2**. Its dataset is the whole trick, and the rule is: it must be a **mixture**, never the RL winners alone.

$$\mathcal{D}_{\text{SFT2}} = \underbrace{\mathcal{D}_{A}^{\text{RL winners}}}_{\text{keep/distill the new } A} + \underbrace{\mathcal{D}_{B,C,D,E}^{\text{general / replay}}}_{\text{restore breadth}}$$

The first term is harvested from RL itself: you keep the **high-reward trajectories** — the proofs that verified, the solutions that passed — and treat them as gold supervised targets. This is exactly the *bootstrapping of verified trajectories* / expert-iteration move from [[code-world-models]]. The second term is **replay** — a slice of the original general/pretraining-style data — whose only job is to say "and keep being good at everything else." Train on the union and the single instruction to the model is:

> Keep the new thing you learned in A, *while* restoring the other capabilities — ideally ending at $A\uparrow, B\uparrow, C\uparrow, D\uparrow, E\uparrow$.

The mixture ratio is a live knob with two failure modes at the extremes. Too heavy on general/replay data (e.g. 90% general $+ 10\%$ RL winners) and SFT2 washes the RL gains back out, $A\uparrow\uparrow \to A\uparrow$. Too heavy on RL winners and SFT2 *narrows* the policy further instead of broadening it — you have just done more specialisation under a supervised label.

Three knobs for preservation: KL, replay loss, post-RL SFT

There are three distinct mechanisms for not-forgetting, and they operate at different points and in different currencies. It is worth separating them cleanly because they are easy to conflate:

KL penalty :	stay behaviourally close to the old policy
replay / SFT loss :	explicitly rehearse the old capabilities on
post-RL SFT (SFT2) :	consolidate new + old after exploration is

The **KL penalty** is the softest. It is added inside the RL objective itself,

$$\mathcal{L} = \mathcal{L}_{\text{RL}} + \beta\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}),$$

and it never mentions B, C, D, E at all. It does not say "solve the old tasks"; it says "don't move your output distribution too far from the reference model π_{ref}." Breadth is preserved *indirectly*, as a side effect of staying near a model that already had breadth. This is the same $\beta\, D_{\text{KL}}$ term that appears in the PPO-style objective in [[code-world-models]], here reread as a forgetting-control device.

The **replay loss** is explicit: you literally include old-data examples and a supervised loss on them, so the model is *scored* on B, C, D, E during training, not merely kept near a model that could do them.

The **post-RL SFT** is the consolidation stage of the previous section — it acts *after* RL rather than during it. Crucially these three are not mutually exclusive; strong systems combine all three.

Approach 1 — fold preservation into RL

The first way to use these knobs is to mix preservation **directly into RL**, so every update simultaneously improves A and rehearses the rest:

$$\mathcal{L} \;=\; \mathcal{L}_{\text{RL}} \;+\; \lambda\, \mathcal{L}_{\text{general}}.$$

The gradient is then a sum of two pulls,

$$\nabla\mathcal{L} \;=\; \nabla\mathcal{L}_{\text{RL}} \;+\; \lambda\, \nabla\mathcal{L}_{\text{general}},$$

and the second term acts as a restoring force: whenever the RL gradient would move the weights in a way that hurts the old abilities, the general-loss gradient pushes back. You prevent drift **as it happens** rather than repairing it later.

Pros. You reduce forgetting *during training itself*, hold broad capability throughout, and may skip a large repair stage entirely.

Cons. The two objectives can fight. If RL wants $\theta \to \theta + \Delta\theta_A$ but the general-SFT term wants $\theta \to \theta - \Delta\theta_A$, the effective step is the partial cancellation $\Delta\theta = \Delta\theta_{\text{RL}} + \lambda\, \Delta\theta_{\text{SFT}}$ — weaker, so slower improvement on the new capability. In the limit λ too large, RL barely changes the model at all. You have bought breadth at the price of depth, and — the subtler cost — at the price of *exploration*, which the next sections make the crux.

Approach 2 — RL first, then consolidation SFT

The second way separates the stages in time:

$$\text{SFT}_1 \;\to\; \text{RL (specialise / discover aggressively)} \;\to\; \text{SFT}_2.$$

RL is allowed to search freely — "forget about preserving everything perfectly for a moment; go find really strong A-strategies" — and may discover several distinct good behaviours A_1, A_2, A_3. Only then do you build the mixture dataset $\mathcal{D}_{\text{SFT2}} = \{A_1, A_2, A_3\} \cup \mathcal{D}_{\text{general}}$ and consolidate the discoveries and the old breadth into one balanced policy.

Pros. RL explores without fighting a general-data gradient on every step, so it can reach stronger behaviours; SFT2 then "compiles" those discoveries into a stable, cheap-to-sample policy *and* restores breadth in one pass.

Cons. The RL checkpoint can drift far before you repair it, and a badly balanced SFT2 can partially undo the RL gains (the mixture-ratio failure modes from above).

Why separating the stages buys exploration

Here is the point that is easy to get backwards. It is tempting to think simultaneous replay (Approach 1) is the one that "keeps the model broad, so it won't collapse into a narrow region." For *preservation* that is true. But for *exploration* the causality runs the other way, and that is precisely why one sometimes prefers $\mathrm{SFT}_1 \to \mathrm{RL} \to \mathrm{SFT}_2$.

With simultaneous replay the update is $\Delta\theta = \Delta\theta_{\mathrm{RL}} + \lambda\,\Delta\theta_{\mathrm{SFT}}$: the RL pull says "go over here, this behaviour earns much more reward," while the SFT pull says "don't move too far, keep doing all the old behaviours." Safer — but if a genuinely strong new strategy lives *far* from the initial SFT policy,

$$\theta_{\mathrm{SFT}} \;\longrightarrow\; \theta^{*}_{\mathrm{new}} \quad (\text{a long way off}),$$

the replay term keeps hauling you back toward θ_{SFT} and you may **never reach** $\theta^{*}_{\mathrm{new}}$. Separating the stages removes that leash *during the search*: RL is free to travel to $\theta^{*}_{\mathrm{new}}$, and only afterwards does SFT2 pull the consolidated model back toward breadth. The logic is "RL gets freedom to explore → SFT2 consolidates what was worth keeping," rather than constraining the explorer on every step.

The catch: SFT2 cannot recover what RL never discovered

Separation is not a free lunch, and this is the sharpest caveat in the chapter. SFT2 can only consolidate trajectories that **actually exist** — the RL winners you harvested. If RL collapses its policy before it explores, those winners were never generated, and no amount of later supervised training conjures them back.

Concretely, suppose A_1, A_2, A_3, A_4 are all viable strategies and A_4 is the best. If RL suffers **entropy collapse** (the diversity-loss failure of [[code-world-models]]) and locks onto, schematically, $P(A_1) \approx 0.999$ early, it may never *sample* A_4 and so never learn it is better. SFT2 can afterwards say "here are the B, C, D, E examples, be broad again" — but it can never say "here is the excellent A_4 trajectory you never discovered." Restoration is possible; *retroactive discovery* is not.

The consequence is that even in the "free RL" phase you still do **not** want unconstrained collapse. You keep the usual exploration regularisers running — a KL term, PPO clipping, diverse prompts, sampling temperature, an entropy

bonus — not to preserve old skills this time, but to keep the policy *exploring* long enough to find A_4 in the first place. The idealised phase is therefore:

$$\mathrm{SFT}_1 \to \underbrace{\text{RL with enough regularisation to keep exploring}}_{\text{but not so much it cannot move}} \to \mathrm{SFT}_2 \text{ (R}$$

Is SFT2 mandatory? The iterative distillation view

No — a separate "recovery SFT" every time is not required. After $\mathrm{SFT}_1 \to \mathrm{RL}$ on $A \to$ harvest high-reward A samples you have two legitimate exits:

1. **Keep the RL checkpoint as-is.** If RL was run with good KL/replay regularisation, the final checkpoint may already be the best production model; no SFT2 needed.
2. **Distill into the next model.** More commonly the RL phase is treated as *search*, and its winners are used to build the *next* model's supervised set rather than to patch the current one in place:

$$M_0 \xrightarrow{\text{RL / search}} \text{good trajectories}, \qquad \mathcal{D}_{\text{new}} = \mathcal{D}_{\text{old SFT}} + \mathcal{D}_{\text{good trajectories}},$$

then $M_1 \xrightarrow{\text{RL again}} M_2$, and so on. This generalises to the full iterative loop frontier labs run:

$$\mathrm{SFT} \to \mathrm{RL} \,/\, \text{search} \to \text{harvest winners} \to \mathrm{SFT} \,/\, \text{distill} \to \mathrm{RL} \to \cdots$$

with KL and replay active throughout to keep each RL leg from destroying what the previous legs built. A subtlety worth internalising: often much of the end-to-end gain is actually realised by the **supervised training on the discovered trajectories**, with RL playing the role of a *discovery engine* that surfaces targets SFT then locks in cheaply.

The tradeoff in one line, and the mental model

Everything above collapses to a single axis:

$$\underbrace{\text{preserve too strongly}}_{\Rightarrow \text{ less new learning}} \qquad \text{versus} \qquad \underbrace{\text{optimise too aggressively}}_{\Rightarrow \text{ forgetting / over-specialisation / entropy c}}$$

The recipe is the art of sitting between them: **enough constraint to avoid collapse, enough freedom to discover genuinely new behaviour.** The clean mental model for the three supervised/RL roles is:

$$\mathrm{SFT}_1 = \textbf{teach} \;\longrightarrow\; \mathrm{RL} = \textbf{explore / optimise} \;\longrightarrow\; \mathrm{SFT}_2 = \textbf{consc}$$

and in strong systems this is not run once but iterated, $\mathrm{SFT} \to \mathrm{RL} \to \mathrm{SFT} \to \mathrm{RL} \to \cdots$.

> $\mathrm{SFT} \to \mathrm{RL} \to \mathrm{SFT}_2$ is usually the sensible default — SFT teaches a strong starting policy, RL explores freely for better behaviour, and a *mixture* consolidation set ($\mathcal{D}_A^{\text{RL winners}} + \mathcal{D}^{\text{general}}$) both distills the win and restores breadth. But it is not automatically best: SFT2 too heavy on general data washes out the RL gain, too heavy on winners narrows the policy further, and — the hard constraint — SFT2 can only consolidate what RL was free enough to discover. Keep KL / replay / entropy regularisation alive during RL not only to remember the old skills but to keep exploring long enough to find the new one.

Chapter 9: Where to Spend Computation — Depth, Chain-of-Thought, Looping, and the Harness

Almost every modern AI system is the *same* object — an autoregressive Transformer $f_\theta(\text{context})$ — made to do almost everything: reason, search, remember, orchestrate tools. This chapter is the synthesis: it separates the **four distinct places** where such a system can spend extra computation when you want it to "think longer," shows why each one has a completely different *hardware* and *memory* signature, and ends with the research question that ties them together — **which computation should happen at which level?** It pulls together the GPU/attention mechanics of [[long-contexts]], the agent scaffold of [[the-agent-harness]], and the discovery-engine view of RL from [[code-world-models]], and reads them as four rungs of one ladder. The framing follows the systems researcher Zhang's argument that we currently mismatch task to machinery almost everywhere.

The four levels where extra computation can live

Stack them from innermost to outermost:

Level 1	Transformer depth	one forward pass through L la
Level 2	chain-of-thought	generate more tokens
Level 3	looped / latent reasoning	more iterations in representat
Level 4	harness / agent / RLM	more model calls, tools, recurs

The key move of the chapter is to stop treating "think longer" as one dial. Each level buys more computation in a different *currency*, and each one stresses the hardware differently — some are **compute-bound**, some are **memory-bandwidth-bound**, some mostly waste wall-clock on coordination. Getting the match right is where the performance is.

Level 1 — Transformer depth, and why decode is memory-bound

A standard Transformer sends a representation through a fixed stack of layers:

$$h \;\to\; F_1 \;\to\; F_2 \;\to\; \cdots \;\to\; F_L.$$

Every output token gets exactly **one** traversal of those L layers. The weights sit permanently in GPU **HBM** (high-bandwidth memory) during inference — but the compute units cannot do arithmetic on HBM-resident weights directly. Each weight tile must travel

$$\text{HBM} \;\rightarrow\; \text{on-chip SRAM / cache} \;\rightarrow\; \text{Tensor Cores}$$

before it can multiply anything. That data-movement path, not the raw FLOP count, is what governs cost — the same memory-hierarchy story that made FlashAttention necessary in [[long-contexts]]. Whether depth is cheap or expensive depends entirely on how many tokens share each weight read.

Prefill is compute-bound. During prefill the whole prompt is processed together,

$$XW, \qquad X \in \mathbb{R}^{N \times d},$$

so one read of W is amortised across all N prompt tokens. High **arithmetic intensity** (FLOPs per byte of weight fetched) keeps the Tensor Cores busy — the GPU is doing what it is good at.

Low-batch decode is memory-bound. Autoregressive decoding emits one token at a time:

$$xW, \qquad x \in \mathbb{R}^{1 \times d}.$$

Now you drag the *entire* parameter matrix out of HBM to produce a single token. The arithmetic intensity is terrible — you move huge amounts of weight data to do very little math — so decode is limited by memory bandwidth, not compute.

Continuous batching is the fix: pack B concurrently-active requests so they share one weight read,

$$xW \;\longrightarrow\; XW, \qquad X \in \mathbb{R}^{B \times d}.$$

Larger B means more FLOPs per byte of weights fetched, pushing decode back toward compute-bound. But at long sequence lengths a *second* memory bottleneck appears: the **KV cache**. The per-layer key/value tensors live mostly in HBM, and every new query must read the relevant historical K/V entries — the GQA/KV-cache pressure analysed in [[long-contexts]]. So Level 1 has two memory walls: the weights (relieved by batching) and the KV cache (which grows with every position).

Level 2 — Chain-of-thought: more tokens, more sequential passes

An ordinary reasoning model buys extra computation the crudest way — by emitting more tokens:

$$r_1, r_2, \ldots, r_T.$$

Each reasoning token makes a *full* trip through all L layers to produce the next:

$$r_t \;\rightarrow\; F_1 \;\rightarrow\; \cdots \;\rightarrow\; F_L \;\rightarrow\; r_{t+1}.$$

So the identity is simply

$$\boxed{\text{more CoT} \;=\; \text{more sequential decode passes}}.$$

This is doubly expensive. Decode is inherently **sequential** (token $t+1$ needs token t) and, per Level 1, each of those passes is bandwidth-limited. Worse, every reasoning token becomes another sequence position, so it *enlarges the KV cache* — the cost of thinking longer at this level compounds in memory as well as time.

Level 3 — Latent / recurrent / looped reasoning

A **looped Transformer** reuses the same block repeatedly, feeding its own output back as input:

$$h^{(r+1)} \;=\; F_\theta\left(h^{(r)}\right).$$

Instead of externalising every intermediate step as an English token, the system manipulates **latent vectors** in place. The contrast with CoT is the crux of the level:

$$\boxed{\text{CoT} \;=\; \text{sequential computation through } \textbf{token} \text{ space}}$$

$$\boxed{\text{looping} \;=\; \text{sequential computation through } \textbf{representation} \text{ space}}$$

Because the loop never emits tokens, it can replace thousands of reasoning tokens with a handful of latent iterations — and crucially avoid the corresponding **KV-cache growth** that Level 2 incurs.

Weight sharing cuts unique storage. Looping also lets you reuse one small block many times rather than storing many distinct layers:

$$60 \text{ unique layers} \;\longrightarrow\; 6 \text{ layers} \times 10 \text{ loops}.$$

But — the honest caveat — this does **not** automatically cut HBM *traffic*. The same recurrent weights may still have to be re-fetched every loop unless enough of them stay resident in cache. Fewer unique parameters stored $\neq$ fewer bytes moved.

Why it can fit GPUs better. If each loop operates over *many* latent positions at once,

$$H \in \mathbb{R}^{N \times d},$$

then recurrent reasoning becomes large, dense matrix multiplications with high arithmetic intensity — much friendlier to Tensor Cores than one-token-at-a-time CoT. The ambition is therefore

$$\boxed{\text{bandwidth-heavy token reasoning} \longrightarrow \text{dense latent FLOPs}},$$

trading a memory-bound workload for a compute-bound one that modern hardware prefers.

Level 4 — Harness, agents, and Recursive Language Models

The outermost level treats the model as a black box $f_\theta(\text{context})$ and asks how *repeated calls* to it are composed — the scaffold studied in [[the-agent-harness]]. A standard coding agent is a loop:

$$\text{LLM} \;\to\; \text{tool} \;\to\; \text{LLM} \;\to\; \text{tool} \;\to\; \cdots$$

Most current harnesses carry the **entire history** forward as the next prompt:

$$C_t = [\,\text{user},\ \text{model actions},\ \text{tool calls},\ \text{tool outputs},\ \ldots\,],$$

and then feed C_t back in — the **trajectory-as-prompt** design. As the trajectory grows enormous, systems resort to **compaction**,

$$\text{large context} \longrightarrow \text{summary},$$

which saves context budget but *loses information* — the context-management tradeoff from [[the-agent-harness]].

RLMs change the memory architecture. A **Recursive Language Model** moves most of the state *out* of the Transformer context and into an external computational environment — files, Python variables, databases, sub-agent outputs — and has the model write *code* to access only what it needs:

$$\{\,\text{files},\ \text{Python variables},\ \text{databases},\ \text{sub-agent outputs}\,\}.$$

```
chunks  = split(document)
answers = [call_agent(chunk) for chunk in chunks]
final   = aggregate(answers)
```

Here code becomes the model's orchestration language. The model can recursively invoke other model calls,

$$A(P) \;\to\; A(P_1),\ A(P_2),\ A(P_3),$$

and those sub-agents may recurse again,

$$A(P_1) \;\to\; A(P_{11}),\ A(P_{12}),$$

— hence *Recursive* Language Model. A system like **Prime Agent** pushes this further by exposing something close to an **IPython environment** as the core interface, rather than handing the model dozens of disconnected top-level tools:

instead of `trajectory as prompt`, the agent reads and writes an external, programmable memory.

The synthesis: which computation belongs at which level

Collecting every form of "thinking longer" into one table:

CoT	generate more tokens
looped transformer	perform more latent iterations
agent	perform more model / tool calls
RLM	write programs + recursively call models
swarm	run many agents / search branches

The research question that organises all of it is:

$$\boxed{\text{which computation should happen at which level?}}$$

The bet is that today's assignments are badly mismatched. Maybe something that now costs 1000 CoT tokens should instead be 20 latent iterations. Maybe something that now takes 20 agent calls could be absorbed into internal routing plus recurrent depth. Maybe a trivial binary decision should not invoke an autoregressive LLM at all. Maybe long-context research should stop holding millions of historical tokens in the prompt and instead manipulate external memory with code — and maybe only genuinely hard search problems deserve an expensive agent **swarm**, which can burn enormous compute if its branches are not coordinated. The overarching point: we force *one architecture* to do everything, even when the task's computation, memory behaviour, and output format are poorly matched to it. The frontier is pulling those assumptions apart and choosing better homes for memory, control flow, recursion, and reasoning.

The one systems equation to keep

If you keep a single equation from this chapter, make it:

$$\boxed{\text{AI performance} \approx \frac{\text{useful computation}}{\text{expensive data movement} + \text{wasted search}}}.$$

Each level attacks a different term of this ratio:

- **GPU kernels** (FlashAttention, fused ops) attack the **data-movement** denominator.
- **Loop transformers** change *how internal compute is allocated* — turning bandwidth-bound token decode into compute-bound latent FLOPs.
- **RLMs / harnesses** reorganise *how repeated model calls, memory, and tools* are composed — shrinking the context you drag around.

- **Agent swarms** attack the **search** side — but, uncoordinated, they inflate the denominator instead, spending compute without buying useful work.

> There is no single "think longer" knob. Extra computation can live in Transformer **depth** (one pass, memory-bound at decode), in **chain-of-thought** (more sequential token passes, growing the KV cache), in a **looped** model (more iterations in representation space — dense, compute-bound, KV-cache-free), or in the **harness** (more model/tool/recursive calls, with RLMs moving state into external code-addressable memory). They have different hardware and memory signatures, so the real lever is *placement* — maximise useful computation per unit of data movement and wasted search by putting each piece of work at the level whose cost structure fits it.

Chapter 10: Scaling Laws — Compute Accounting, Chinchilla, and the Training–Inference Trade-off

Chapter 2 touched scaling laws in one short section ([[code-world-models]]); this chapter is the full treatment. The single most important idea to hold onto up front is this: **scaling laws are empirical models, not laws of nature.** They are power-law curves *fitted* to the results of hundreds of training runs, and they earn their keep by letting you *predict* the loss of a run you have not yet paid for and *allocate* a compute budget intelligently. They do not guarantee how any particular architecture must behave, and they extrapolate at your own risk. With that caveat fixed, the chapter builds the machinery from the ground up: first the compute-accounting identity $C \approx 6ND$ and exactly where every factor comes from, then the Chinchilla loss model and how it is fitted, then the calculus that turns it into a compute-optimal recipe, and finally the question the session kept circling back to — *if training a model longer always helps, what exactly do you give up by doing it?*

The five equations to keep

Everything below is in service of these five:

$C_{\text{train}} \approx 6ND$	dense-Transformer training FLO
$L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$	Chinchilla empirical loss model
$N_{\text{opt}} \propto C^{\,\beta/(\alpha+\beta)}$	compute-optimal parameter scali
$D_{\text{opt}} \propto C^{\,\alpha/(\alpha+\beta)}$	compute-optimal data scaling
$C_{\text{lifetime}} \approx 6ND + 2NT_{\text{out}}$	training + lifetime inference

with N = parameters, D = tokens *processed* during training (repeats included), C = FLOPs, L = held-out loss, E = fitted limiting loss, α, β = fitted exponents, A, B = fitted coefficients, and T_{out} = tokens the deployed model will ever generate.

Where $6ND$ comes from

Take a single linear layer — the atom every Transformer is built from:

$$y = Wx, \qquad W \in \mathbb{R}^{m \times n}, \qquad N = mn.$$

Forward pass. Each of the m outputs is a dot product of length n: n multiplies and $n - 1$ adds, i.e. $\approx 2n$ FLOPs. Across all m outputs that is $\approx 2mn = 2N$ FLOPs per input vector.

Backward pass does *two* distinct matrix multiplies. Writing $g = \partial L/\partial y$ for the gradient arriving from above:

$$\frac{\partial L}{\partial x} = W^\top g \qquad \text{(pass the gradient to the previous layer)},$$

$$\frac{\partial L}{\partial W} = g\, x^\top \qquad \text{(tell this layer how to update its weights)}.$$

For a batch of B token positions these become $Y = WX$, $dX = W^\top G$, $dW = GX^\top$, and the FLOP ledger per layer is:

operation	**FLOPs per batch**
forward WX	$2NB$
input-gradient $W^\top G$	$2NB$
weight-gradient $GX^\top$	$2NB$
total	**6NB**

Why does the weight gradient cost $2NB$? Each weight accumulates one contribution per example,

$$\frac{\partial L}{\partial W_{ij}} = \sum_{b=1}^{B} G_{ib}\, X_{jb},$$

which is B multiplies and $B-1$ adds $\approx 2B$ FLOPs per weight, and there are N weights. (For a *single* example the weight-gradient outer product is only N multiplies — the clean $2N$-per-token figure is the large-batch matrix-multiply approximation.) If training processes D tokens total in batches of B:

$$\frac{D}{B} \cdot 6NB = \boxed{6ND}.$$

So the famous constant decomposes as **2 forward + 4 backward** FLOPs per parameter per token.

Why input gradients are computed even though only weights update

A natural objection: we only ever *update* weights, so why spend two-thirds of the backward budget computing $\partial L/\partial x$? Because backprop is the chain rule run **once, outside-in**, over a deep stack:

$$h_\ell = W_\ell\, h_{\ell-1}.$$

Layer ℓ needs the gradient g_ℓ flowing into it to compute *both*

$$dW_\ell = g_\ell\, h_{\ell-1}^\top \qquad \text{and} \qquad g_{\ell-1} = W_\ell^\top g_\ell.$$

That second quantity is precisely the gradient the *next-lower* layer needs to compute its own $dW_{\ell-1}$. The input-gradient multiply is what carries the chain rule down to the layer below; without it, every earlier layer would be blind. The network does **one** backward traversal, not a separate pass per layer, so the cost is $\approx 4N_{\text{total}}$ per token for the whole stack, giving $\approx 6N_{\text{total}}$ total. The only layer that may skip the input gradient is the very first, whose input is raw data with nothing beneath it to update.

> Backprop is strictly more expensive than the forward pass — roughly $2\times$ — because it does two jobs at once: it computes every parameter's gradient *and* propagates the chain rule to the layers underneath.

Why $6ND$ is only an approximation: projections vs attention mixing

$6ND$ counts only the *learned matrix multiplies*. A real Transformer also does attention mixing, normalisation, and activations. The one that actually matters at scale is attention mixing, because it scales with sequence length T in a way the projections do not. For one layer of width d and sequence length T:

operation	forward FLOPs
Q,K,V,output projections	$8Td^2$
MLP projections	$16Td^2$
attention scores $QK^\top$	$2T^2d$
attention-weighted values	$2T^2d$
total	$24Td^2 + 4T^2d$

The learned projections hold $N_{\text{layer}} \approx 12d^2$ parameters, so $2TN_{\text{layer}} \approx 24Td^2$ — exactly the projection term, the part $6ND$ captures. The leftover $4T^2d$ is attention mixing, which carries *no* parameters and so is invisible to $6ND$. Their ratio is clean:

$$\boxed{\frac{C_{\text{attention mixing}}}{C_{\text{learned projections}}} \approx \frac{T}{6d}}$$

For $d = 4096$:

context T	attention mixing as % of projections
2,048	8.3%
8,192	33.3%
32,768	133.3%

So at moderate context the learned-matrix work dominates and $6ND$ is a fine first-order estimate; at very long context attention mixing becomes comparable or larger and $6ND$ undercounts. Normalisation is $O(Td)$, negligible against

the $O(Td^2)$ projections at large d. And none of this is runtime — wall-clock also depends on memory bandwidth, the attention kernel, GPU utilisation, and communication, the whole memory-hierarchy story of [[long-contexts]].

> Always separate **FLOPs** from **runtime**. $6ND$ is a budgeting tool for first-order scaling, not a performance model; serious experiments account for the real architecture and the real hardware.

What a scaling law actually predicts

The general shape fitted in every scaling study is a shifted power law:

$$\boxed{L(X) = E + AX^{-\alpha}}, \qquad \alpha > 0,$$

where X is parameters, data, or compute, E is the limiting loss the fit implies, A sets the scale of the *reducible* loss, and α is the rate of improvement. The exponent is negative because

$$\frac{d}{dX}AX^{-\alpha} = -\alpha A X^{-\alpha-1} < 0, \qquad \frac{d^2}{dX^2}AX^{-\alpha} = \alpha(\alpha+1)AX^{-\alpha-2} >$$

i.e. loss falls but with **diminishing returns**: each doubling buys less than the last.

Doubling only shrinks the excess. The subtlety that trips people up is that a constant *fractional* improvement applies to $L - E$, not to L:

$$\frac{L(2N) - E}{L(N) - E} = \frac{A(2N)^{-\alpha}}{AN^{-\alpha}} = 2^{-\alpha}.$$

For $\alpha = 0.3$, $2^{-0.3} \approx 0.812$ — the *excess* loss drops $\approx 18.8\%$ per doubling. But total loss drops less because E never moves. Worked example: $E = 1$, $L(N) = 2$, so excess $= 1$. After doubling, excess $= 0.812$, so $L(2N) = 1.812$ — total loss fell only 9.4%, not 18.8%.

Why log-log plots. Subtract E and take logs:

$$\log(L - E) = \log A - \alpha \log N,$$

a straight line of slope $-\alpha$. That is why scaling curves are drawn log-log and why a clean straight line is the signature of a power law. If a $10\times$ increase halves the excess loss, then $10^{-\alpha} = 0.5$, so $\alpha = -\log_{10} 0.5 \approx 0.301$ — and note this is *measured from the slope*, not predicted by any theory.

The Chinchilla loss function and the meaning of E

Hoffmann et al. (2022, "Chinchilla") fitted the two-variable version:

$$\boxed{L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}}$$

with approximate fitted values

$$E = 1.69, \quad A = 406.4, \quad B = 410.7, \quad \alpha = 0.34, \quad \beta = 0.28$$

(coefficients are tied to the paper's units and setup — do not treat them as universal). Read the three terms as: a **limiting loss** E, a **capacity penalty** $AN^{-\alpha}$ that shrinks as the model gets bigger, and a **data penalty** $BD^{-\beta}$ that shrinks as it sees more tokens. The additive, separable form is an *approximation that fits well*, not a theorem — a more general model could add an interaction term $\gamma N^{-p} D^{-q}$; the simple form is kept because it predicts the measurements.

What E means. Language is genuinely uncertain, so even a perfect next-token predictor has nonzero expected cross-entropy. For a true distribution p and model q,

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q),$$

and the irreducible conditional entropy $H(p)$ motivates a loss floor. But the *fitted* E is not the true entropy — it is whatever constant best straightens the curve over the measured regime, and it can extrapolate wrongly outside it.

How scaling-law experiments are actually run

The equation is the easy part; the experimental controls are where the science lives. The workflow Chinchilla-style studies follow:

1. **Fix a model family and recipe** — architecture, tokenizer, data pipeline, evaluation objective — so runs are comparable. (e.g. dense decoder-only Transformers varying width and depth.)
2. **Choose a grid of budgets** — e.g. $N \in \{100\text{M}, 300\text{M}, 1\text{B}, 3\text{B}, 10\text{B}\}$ crossed with several token budgets each, yielding triples (N_i, D_i, L_i).
3. **Tune optimisation at every scale.** The goal is the best *achievable* loss per scale, $L^{\backslash *}(N, D) = \min_{h \in H} L(N, D, h)$ over learning rate, warmup, batch size, and optimizer settings h — otherwise you are fitting the loss of poorly-tuned models, not of the scale.
4. **Monitor training quality** — train/held-out curves, gradient norms $\|\nabla_\theta L\|_2 = \left(\sum_j (\partial L / \partial \theta_j)^2\right)^{1/2}$, update-to-weight ratios, NaNs/Infs, loss spikes, seed reproducibility. A large or small gradient norm is not automatically failure — the *pattern* matters.
5. **Fit the law** by least squares in log space, $\min_{E,A,B,\alpha,\beta} \sum_i \left(\log \hat{L}_i - \log L_i\right)^2$.
6. **Validate out-of-sample.** Hold out model sizes / budgets, predict their loss, then actually train them and compare. A fit that only looks good on its own

training points proves nothing.

How many runs? There are five free parameters, but five observations is nowhere near enough — a pilot might be 5×5 with tuning and replication, and Chinchilla used **more than 400 runs**. The count depends on noise, the extrapolation range wanted, and how stable the fitted exponents turn out to be.

> A scaling law is only as trustworthy as the compute accounting, optimisation, data quality, and out-of-sample tests behind it. The math optimises resources *after* you have measured a reliable relationship; it cannot rescue a sloppy experiment.

Deriving compute-optimal N and D

Now the payoff: given a fixed training budget $C = 6ND$, how do you split it between model size and tokens? Minimise L subject to the constraint. Solve the constraint, $D = C/(6N)$, and substitute:

$$L(N) = E + AN^{-\alpha} + B\Big(\frac{6N}{C}\Big)^{\beta}.$$

Set $dL/dN = 0$:

$$-\alpha A N^{-\alpha-1} + \beta B\,(6/C)^{\beta} N^{\beta-1} = 0 \implies N^{\alpha+\beta} = \frac{\alpha A}{\beta B}\Big(\frac{C}{6}\Big)^{\beta}.$$

Hence the two scaling exponents fall out directly, and $D = C/(6N)$ gives the other:

$$\boxed{N_{\text{opt}} \propto C^{\,\beta/(\alpha+\beta)}}, \qquad \boxed{D_{\text{opt}} \propto C^{\,\alpha/(\alpha+\beta)}}.$$

The two exponents must sum to 1 because their product ND has to scale like C. Plugging in Chinchilla's $\alpha = 0.34,\ \beta = 0.28$:

$$N_{\text{opt}} \propto C^{0.452}, \qquad D_{\text{opt}} \propto C^{0.548}.$$

The paper's other estimation methods landed near the clean, memorable summary $N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$ — i.e. **scale parameters and tokens roughly in lockstep.** Note that "scale together" is a statement about the *exponents on* C; it does **not** say $\alpha = \beta$.

The "20 tokens per parameter" rule

The folklore ratio

$$\boxed{\frac{D}{N} \approx 20}$$

was *calibrated*, not assumed. The experiment: pick a budget C; train several configs all satisfying $6ND = C$; measure held-out loss; keep the lowest; repeat across budgets; see how the winning N, D move. If the optimum at one budget is $(1\text{B},\ 20\text{B})$ and at another is $(10\text{B},\ 200\text{B})$, both give $D/N = 20$, and the ratio holds across the sweep. Chinchilla's headline model made the point by example:

$$N = 70\text{B}, \quad D = 1.4\text{T} \implies \frac{1.4\text{T}}{70\text{B}} = 20,$$

and it beat the much larger, under-trained Gopher (280B) at comparable training compute. The exact ratio is not fixed by nature — change the architecture, dataset, or objective and it moves.

Kaplan (2020) vs Chinchilla (2022)

Kaplan's earlier analysis had estimated

$$N_{\text{opt}} \propto C^{0.73}, \qquad D_{\text{opt}} \propto C^{0.27},$$

which says "spend new compute mostly on *bigger* models." Chinchilla's $\approx C^{0.5}/C^{0.5}$ says "split evenly." Over a $100\times$ compute increase the two recipes diverge sharply:

	Kaplan	**Chinchilla**
parameters	$28.8\times$	$10\times$
training tokens	$3.47\times$	$10\times$

Later work ("Reconciling…", "Resolving Discrepancies…", 2024) traced the gap to **methodology**, not nature: how embedding parameters were counted in small models, compute for the output-vocabulary projection, learning-rate warmup length, and scale-dependent optimizer tuning. The durable lesson survived both papers — loss follows power laws — while the contested part was the *optimal allocation*, and it turned on experimental bookkeeping. This is the cautionary tale behind the whole chapter: the exponents are artifacts of the controls as much as of the physics.

Fixed compute: what you lose by training longer

This is the question the session kept returning to. *If I just keep training the same model on more tokens, the loss keeps dropping — so what exactly am I losing?*

Start with what Chinchilla optimises. At a fixed budget, say $C = 6 \times 10^{21}$ FLOPs, these three configs all cost the same:

N	D	C
1B	1T	6×10^{21}
5B	200B	6×10^{21}
10B	100B	6×10^{21}

and the law predicts which wins. Suppose the measured losses are 2.5, 2.2, 2.4 — the 5B config is optimal *for this budget*. Now train that 5B model from 200B to 400B tokens:

$$C = 6\,(5 \times 10^9)(400 \times 10^9) = 1.2 \times 10^{22}\ \text{FLOPs},$$

double the original. Its loss might improve $2.2 \to 2.1$. So yes — more training helped. The trap is the counterfactual: *with that doubled budget, could a different N, D split have done even better?*

config	**compute**	**loss (illustrative)**
5B, 200B	$1\times$	2.20
5B, 400B	$2\times$	2.10
7B, 286B	$2\times$	2.05

The 7B config spends the *same* doubled compute and reaches a lower loss. That is the precise meaning of **training-compute suboptimal**: you did not lose the improvement you got, you lost the *larger improvement the same compute could have bought* had you also grown the model. Said the way the session finally put it: you could have had a **bigger model with lower loss** for that compute — but its **inference would then cost more too**. That trade is the whole next section.

Processed tokens vs unique tokens

A second, independent subtlety hides in the symbol D. Training on 200B unique tokens for one epoch and training on 200B unique tokens for *two* epochs both set $D_{\text{processed}} = 400\text{B}$ of compute — but they are not the same experiment:

$$D_{\text{processed}} \neq D_{\text{unique}}.$$

A second epoch can still lower loss (more optimiser steps over the data), but repeated data does **not** carry the same information as genuinely new data, and past a point repetition yields diminishing returns or outright overfitting. The Chinchilla law was calibrated in a largely single-pass, fresh-data regime, so it must **not** be blindly extrapolated to heavy repetition of a fixed corpus. Keep the two D's mentally distinct whenever you reason about "training longer."

Inference-aware scaling — Beyond Chinchilla

Chinchilla answers exactly one question:

$$\textbf{Objective A:} \quad \min_{N,D} L(N, D) \quad \text{s.t.} \quad 6ND \le C$$

"*Given this pretraining budget, what is the lowest validation loss?*" But a company that will serve the model cares about the **whole lifecycle**. For dense inference, parameter-associated compute is $\approx 2N$ FLOPs per output token, so over a lifetime output of T_{out} tokens:

$$\boxed{C_{\text{lifetime}} \approx \underbrace{6ND}_{\text{train}} + \underbrace{2N\,T_{\text{out}}}_{\text{serve}}}$$

and the real objective becomes:

$$\textbf{Objective B:} \quad \min_{N,D} \left(6ND + 2N\,T_{\text{out}}\right) \quad \text{s.t.} \quad L(N, D) \le L_{\text{target}}.$$

Why does this favour a *smaller* model? Because inference cost scales with N alone, and a smaller model trained on far more tokens can reach a target quality while costing less on every one of the billions of tokens it will later generate. A hypothetical comparison at equal quality:

	large	**smaller, longer-trained**
parameters N	20B	5B
training tokens D	400B	4T
training FLOPs $6ND$	4.8×10^{22}	1.2×10^{23}
inference FLOPs/token $2N$	40B	10B

The smaller model costs $\sim 2.5\times$ *more* to train but $\sim 4\times$ *less* per generated token. Whether that is worth it is set entirely by T_{out} — the more the model is used, the more the one-time training premium pays for itself. This is why production models are deliberately **overtrained relative to the Chinchilla optimum** (the same point Chapter 2 raised from the CWM side — [[code-world-models]]). Three clarifications that matter:

- This is **overtraining, not overfitting** — it is a deliberate budget choice, not a generalisation failure.
- It is distinct from **inference-time** scaling (chain-of-thought, search, looping — [[where-to-spend-computation]]): that spends *extra* compute per answer at serve time; this makes the *base model* cheaper per token.
- Extra training does **not** make an *unchanged* model cheaper to serve. It only lets a *smaller* model become good enough to replace a larger one. And if you already own a trained model, continuing it is often the pragmatic move — Chinchilla compares configs when planning *from scratch*, not a reason to throw away a paid-for model every time your budget grows.

The three questions you must not conflate

The entire training-vs-inference discussion collapses into three separate questions with three separate answers:

question	answer
Will training the same model longer improve quality?	Usually yes (
Is that the best use of the extra compute?	Not necessar
Can longer training make deployment cheaper?	Indirectly ye

The one line to carry out of the chapter:

more training can improve quality $\neq$ best possible quality per FLO

> Scaling laws are fitted power laws, not guarantees. $6ND$ accounts for training FLOPs (2 forward + 4 backward per parameter per token) and undercounts attention once context grows past $\sim 6d$. Chinchilla's $L = E + AN^{-\alpha} + BD^{-\beta}$, fitted over 400+ runs, says scale N and D together (≈ 20 tokens per parameter) to minimise loss at a **fixed training budget** — and the moment you train that model longer you keep the quality gain but forfeit the larger gain a *bigger* model would have bought for the same compute, buying a lower inference cost instead. Which direction is "optimal" depends entirely on whether you are minimising training FLOPs, lifetime serving cost ($+ 2NT_{\text{out}}$), or the cost of improving a model you already own.